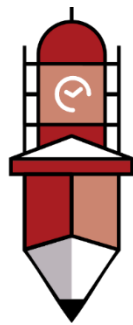


Lab Manual

**Data Mining
DS-303**



**The
University of
Faisalabad**

By

Muhammad Ahmad Mustafa

2023-BSAI-016

Submitted to

Sir Arham

Bachelor of Science in Artificial Intelligence

DEPARTMENT OF COMPUTER SCIENCES

Table Of Contents

Lab 1: KNIME Introduction.....	3
Lab 2: Customer Churn Prediction.....	11
Lab 3: Data Cleaning.....	17
Lab 4: Data Preprocessing	23
Lab 5: Market Basket Analysis.....	30
Lab 6: Apriori Algorithm.....	38
Lab 7: FP-Growth	44
Lab 8: Python Basics.....	50
Lab 9: Decision Tree.....	56
Lab 10: Naïve Bayes & KNN.....	63
Lab 11: SVM	70
Lab 12: Clustering.....	77
Lab 13: Outlier Detection.....	86
Lab 14: Scraping and Sentiment Analysis.....	92
Lab 15: Final Capstone.....	96

Lab # 1

KNIME Analytics for Retail Sales Evaluation

Business Case Overview

A retail organization maintains a large collection of sales transaction data covering multiple product categories. Although the information is available, it cannot effectively support business decisions unless it is properly processed and analyzed. Managers need meaningful insights to evaluate total sales performance, calculate the average spending of customers, identify the most profitable product categories, and determine which products contribute the highest share of revenue.

This practical exercise demonstrates how the KNIME Analytics Platform can be used to transform raw retail sales records into actionable business insights through a structured workflow-based approach.

Overview of the Dataset

The dataset used in this lab is a Retail Sales CSV file that contains detailed information about sales transactions.

Each record represents a completed purchase and includes the following attributes:

- Transaction ID
- Product Name
- Product Category
- Quantity Purchased
- Unit Price
- Date of Purchase
- Sales Revenue (generated during data processing)

This dataset is ideal for performing sales summarization, revenue calculations, category-wise comparisons, and product performance analysis.

Analytical Tool Utilized

KNIME Analytics Platform

KNIME is a visual data analytics tool that enables users to build workflows by connecting different processing nodes. Each node performs a specific operation such as importing datasets, cleaning data, converting data types, applying calculations, generating statistical summaries, or aggregating information.

By linking these nodes together, users can create a complete analytical workflow that converts raw sales data into meaningful information for business decision-making.

Expected Learning Outcomes

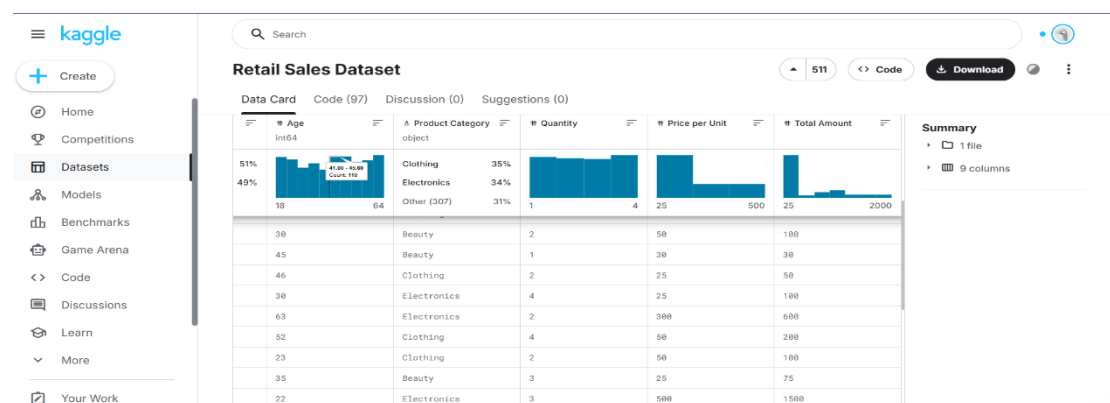
Upon completion of this activity, students will be able to:

1. Import a retail sales dataset into KNIME using the CSV Reader node.
2. Examine and configure data types to ensure accurate analysis.
3. Generate descriptive statistics to gain an overview of the dataset.
4. Create a Revenue column using the Math Formula node.
5. Evaluate key retail performance indicators (KPIs) using GroupBy and Sorter nodes.

Implementation Steps

Step 1: Acquire the Dataset

Download the Retail Sales dataset in CSV format from the provided source and save it on your computer before starting the analysis process.



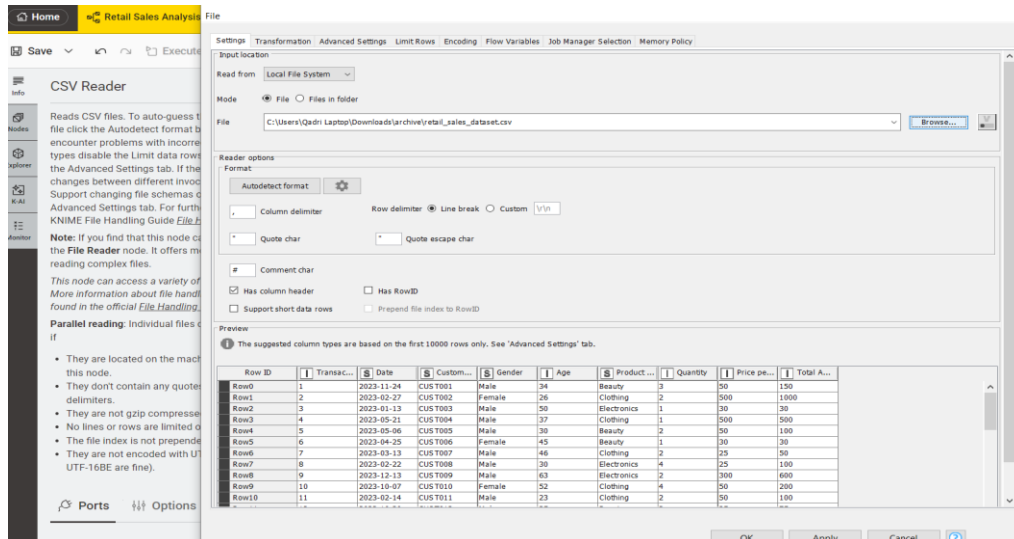
Step 2: Develop a New Workflow

Open KNIME Analytics Platform and create a new workflow titled **Retail Sales Analytics**. This workflow will serve as the workspace for all data preparation and analysis tasks.

The screenshot shows the 'Create a new workflow' dialog box in KNIME. The 'Workflow name' field is filled with 'Retail Sales Analysis'. There are 'Cancel' and 'Create' buttons at the bottom right.

Step 3: Load the Sales Dataset

Insert a **CSV Reader** node into the workflow and import the Retail Sales CSV file. Execute the node and confirm that the dataset has been loaded correctly.



Step 4: Validate Data Types

Review the imported columns to ensure that numerical fields such as **Quantity**, **Price**, and **Revenue** are assigned appropriate numeric data types (Integer or Double), while fields like **Product Category** and **Product Name** are stored as String values.

► 1: File Table Flow Variables										
Rows: 1000 Columns: 9										
Table Statistics										
	#	RowID	Transacti...	Date	Customer	Gender	Age	Product C...	Quantity	Price per ...
	1	Row0	1	2023-11-24	CUST001	Male	34	Beauty	3	50
	2	Row1	2	2023-02-27	CUST002	Female	26	Clothing	2	500
	3	Row2	3	2023-01-13	CUST003	Male	50	Electronics	1	30
	4	Row3	4	2023-05-21	CUST004	Male	37	Clothing	1	500
	5	Row4	5	2023-05-06	CUST005	Male	30	Beauty	2	50
	6	Row5	6	2023-04-25	CUST006	Female	45	Beauty	1	30

Step 5: Perform Descriptive Analysis

Use the **Statistics** node to generate key descriptive measures including average, minimum value, maximum value, total sum, and standard deviation for the numerical columns.

Table 1: Statistics Table

#	RowID	Column	Min	Max	Mean	Std. devia...	Variance	Skewness	Kurtosis	Overall su...	No. n
1	Transa	Transaction ID	1	1,000	500.5	288.819	83,416.667	0	-1.2	500,500	0
2	Age	Age	18	64	41.392	13.681	187.182	-0.049	-1.201	41,392	0
3	Quantit	Quantity	1	4	2.514	1.133	1.283	-0.014	-1.393	2,514	0
4	Price p	Price per Unit	25	500	179.89	189.681	35,979.017	0.736	-1.139	179,890	0
5	Total A	Total Amount	25	2,000	456	559.998	313,597.347	1.376	0.815	456,000	0

Step 6: Create a Revenue Attribute

Add a **Math Formula** node and generate a new column named **Revenue** by applying the following calculation:

$$\text{Revenue} = \text{Quantity} \times \text{Price}$$

The newly created field will be used throughout the subsequent KPI analysis.

Dialog - 3:3 - Math Formula

Math Expression

Column List

- Transaction ID
- Age
- Quantity
- Price per Unit
- Total Amount

Function

- ROWCOUNT
- ROWINDEX
- pi
- e
- COL_MIN(col_name)
- COL_MAX(col_name)
- COL_MEAN(col_name)
- COL_MEDIAN(col_name)
- COL_SUM(col_name)
- COL_STDEV(col_name)
- COL_VAR(col_name)
- ln(x)
- log(x)
- log10(x)
- exp(x)

Expression

1. Quantity * Price per Unit

Append Columns: new column

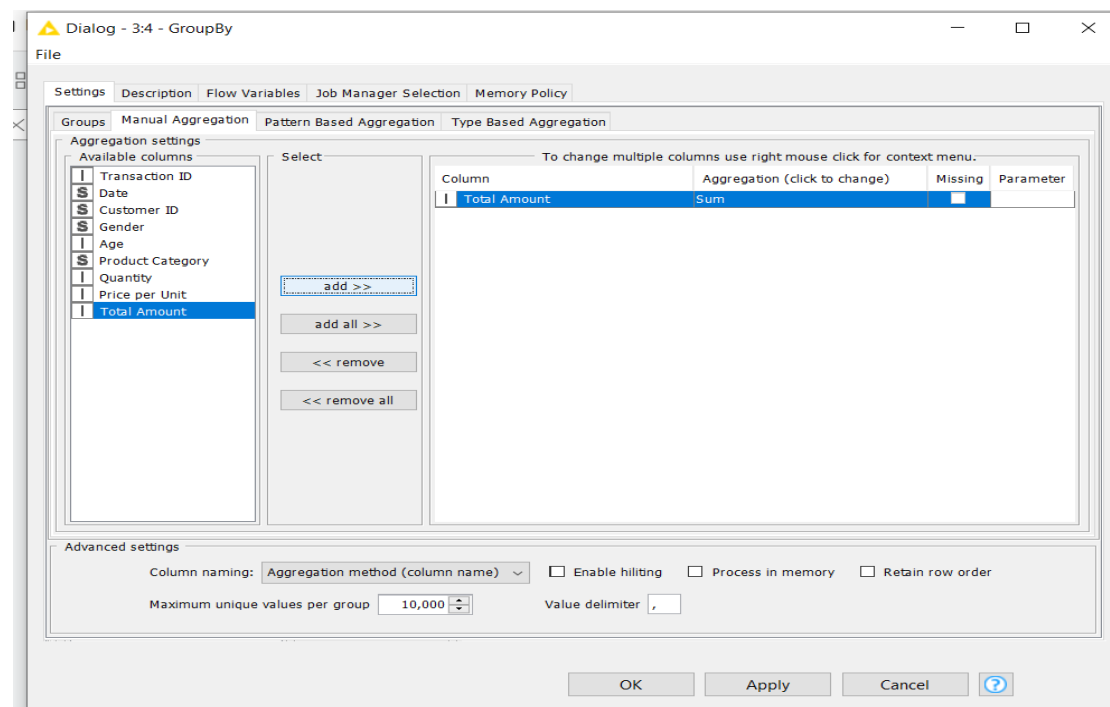
Replace Column: Total Amount

Convert to Int

OK Apply Cancel ?

Step 7: Determine Overall Sales Revenue (KPI 1 – Total Sales Revenue)

Connect a **GroupBy** node without specifying any grouping field. Apply the **Sum** aggregation method on the Revenue column to calculate the total revenue generated from all sales transactions.



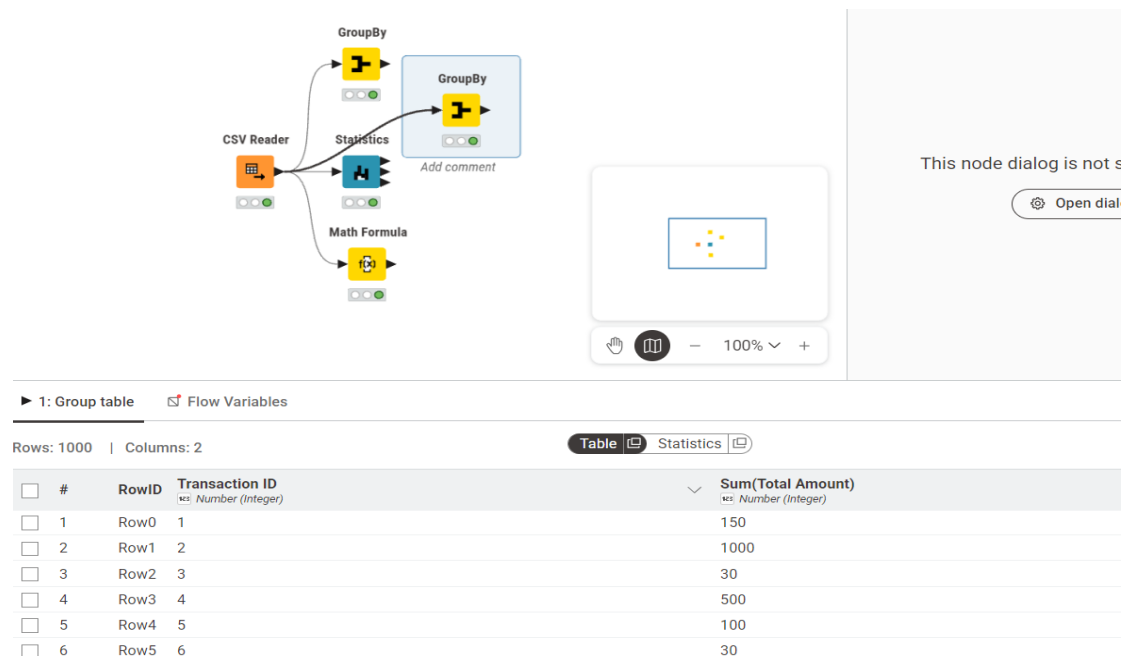
Step 8: Review Total Revenue Output

Execute the node and examine the resulting value, which represents the cumulative sales revenue of the entire dataset.

► 1: Group table Flow Variables			
Rows: 1 Columns: 1			
Table Statistics			
☐	#	RowID	Sum(Total Amount) Number (Integer)
☐	1	Row0	456000

Step 9: Compute Revenue for Each Transaction

Insert another **GroupBy** node and group the data by **Transaction ID**. Aggregate the Revenue column using the **Sum** function to calculate revenue generated by each individual transaction.



1: Group table

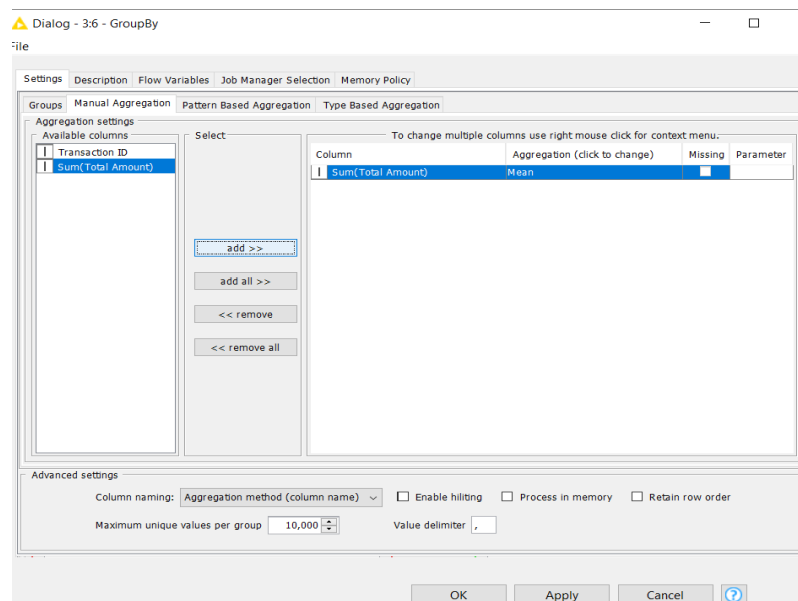
Flow Variables

Rows: 1000 | Columns: 2

#	RowID	Transaction ID	Sum(Total Amount)
1	Row0	1	150
2	Row1	2	1000
3	Row2	3	30
4	Row3	4	500
5	Row4	5	100
6	Row5	6	30

Step 10: Calculate Average Transaction Value (KPI 2 – Average Order Value)

Connect a second **GroupBy** node without selecting a grouping column. Apply the **Mean** aggregation function to the transaction revenues to determine the average value of customer orders.



Dialog - 3:6 - GroupBy

file

Settings Description Flow Variables Job Manager Selection Memory Policy

Groups Manual Aggregation Pattern Based Aggregation Type Based Aggregation

Aggregation settings

Available columns

- Transaction ID
- Sum(Total Amount)

Select

add >>

add all >>

<< remove

<< remove all

To change multiple columns use right mouse click for context menu.

Column	Aggregation (click to change)	Missing	Parameter
Sum(Total Amount)	Mean		

Advanced settings

Column naming: Aggregation method (column name)

Maximum unique values per group: 10,000

Value delimiter: ,

Enable hilling Process in memory Retain row order

OK Apply Cancel ?

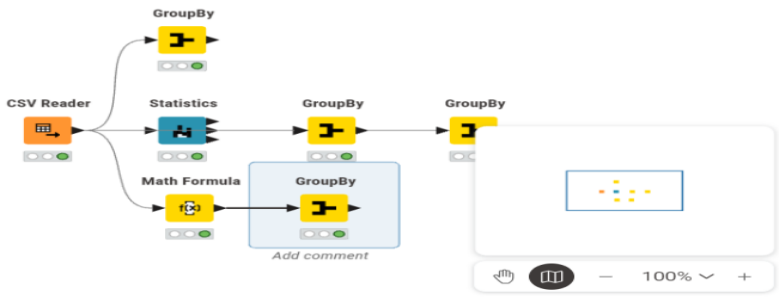
Step 11: Examine Average Order Value

Run the node and inspect the output to view the average revenue earned per transaction.

► 1: Group table ✉ Flow Variables			
Rows: 1 Columns: 1			
☐	#	RowID	Mean(Sum(Total Amount)) .00 Number (Float)
☐	1	Row0	456

Step 12: Analyze Revenue by Product Category (KPI 3 – Category Performance)

Use a **GroupBy** node and group records according to **Product Category**. Aggregate the Revenue field using the **Sum** function to calculate total revenue generated by each category.



► 1: Group table ✉ Flow Variables			
Rows: 3 Columns: 2			
☐	#	RowID	Product Category [1] String
☐	1	Row0	Beauty
☐	2	Row1	Clothing
☐	3	Row2	Electronics

Table 📊 Statistics 📄	
	Sum(Total Amount) [1] Number (Integer)
Beauty	143515
Clothing	155580
Electronics	156905

Step 13: Rank Categories by Revenue

Attach a **Sorter** node and configure it to sort the aggregated revenue values in descending order. This arrangement helps identify the highest-performing categories based on sales revenue.

Sorter

Sorting

Criterion 1

Column: Sum(Total Amount)

Order: ☒ Ascending ☐ Descending

[Add sorting criterion](#)

[Discard](#) [Apply and Execute](#)

1: Sorted Table [Flow Variables](#)

Rows: 3 | Columns: 2

#	RowID	Product Category	Sum(Total Amount)
1	Row0	Beauty	143515
2	Row1	Clothing	155580
3	Row2	Electronics	156905

Lab Outcome

This practical exercise illustrates how transactional retail data can be transformed into meaningful business information through a structured KNIME workflow.

The analytical process follows the sequence:

Sales Records → Data Processing → KPI Computation → Business Intelligence

By utilizing KNIME's node-based environment, users can generate valuable performance indicators that facilitate revenue analysis, category evaluation, and informed decision-making regarding product performance.

Lab # 2

Customer Churn Prediction Using Orange Data Mining

Real-World Problem

Customer retention is a critical challenge for online businesses. An e-commerce company aims to identify customers who are likely to discontinue using its services so that preventive measures can be taken before customer loss occurs. Failure to detect churn early can result in reduced revenue and missed retention opportunities. This practical exercise applies the CRISP-DM framework and Orange Data Mining to develop a predictive model capable of identifying potential customer churn.

Dataset Overview

The analysis utilizes the **Churn_Modelling.csv** dataset, which contains information for 10,000 customers and 14 variables. The target attribute is **Exited**, where:

- **1 = Customer Churned**
- **0 = Customer Retained**

Important predictor variables include:

- CreditScore
- Geography
- Gender
- Age
- Tenure
- Balance
- NumOfProducts
- HasCrCard
- IsActiveMember
- EstimatedSalary

During data preparation, the columns **RowNumber**, **CustomerId**, and **Surname** were excluded because they do not contribute to churn prediction.

Software Platform

Orange Data Mining

Orange is a visual data mining and machine learning platform that enables users to create analytical workflows through drag-and-drop widgets. The software supports data preprocessing, predictive modeling, visualization, and model evaluation without requiring programming knowledge.

Practical Objectives

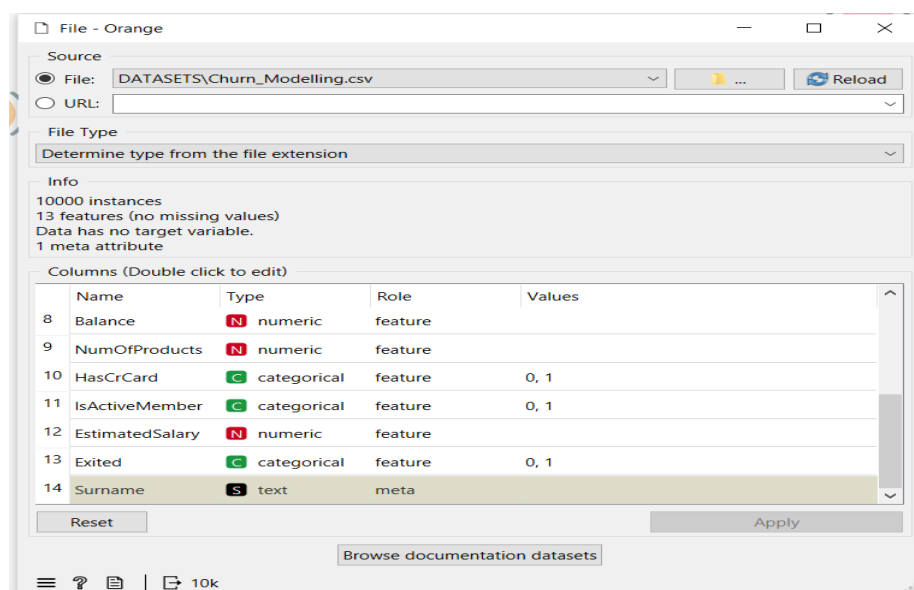
Upon completion of this lab, students will be able to:

1. Import and explore a CSV dataset using File and Data Table widgets.
2. Configure input and target variables using the Select Columns widget.
3. Apply feature scaling and preprocessing techniques.
4. Build classification models using Logistic Regression and Random Forest.
5. Compare model performance using cross-validation techniques.
6. Interpret classification results through ROC curves and confusion matrices.
7. Measure customer churn and retention performance indicators.

Experimental Procedure

Step 1: Import Dataset of Customer

Use the **File** widget to load the *Churn_Modelling.csv* dataset. Connect it to a **Data Table** widget to verify that all records and attributes have been imported successfully.



Data Table - Orange

Info
10000 instances (no missing data)
13 features
No target variable.
1 meta attribute

Variables
☒ Show variable labels (if present)
☐ Visualize numeric values
☒ Color by instance classes

Selection
Clear Selection
☒ Select full rows

Restore Original Order
☒ Send Automatically

	Gender	Age	Tenure	Balance	NumOfProducts
1	Female	42	2	0.00	
2	Female	41	1	83807.86	
3	Female	42	8	159660.80	
4	Female	39	1	0.00	
5	Female	43	2	125510.82	
6	Male	44	8	113755.78	
7	Male	50	7	0.00	
8	Female	29	4	115046.74	
9	Male	44	4	142051.07	
10	Male	27	2	134603.88	
11	Male	31	6	102016.72	
12	Male	24	3	0.00	
13	Female	34	10	0.00	
14	Female	25	5	0.00	
15	Female	35	7	0.00	
16	Male	45	3	143129.41	
17	Male	58	1	132602.88	
18	Female	24	9	0.00	

10k 10k 10k

Step 2: Configure Feature Selection

Attach a **Select Columns** widget to the workflow. Move **RowNumber**, **CustomerId**, and **Surname** to the Ignore section. Assign **Exited** as the target variable and retain all relevant predictive features.

Select Columns - Orange

Ignored (3)
Filter
☒ RowNumber
☒ CustomerId
☒ Surname

Features (10)
Filter
☒ CreditScore
☒ Geography
☒ Gender
☒ Age
☒ Tenure
☒ Balance
☒ NumOfProducts
☒ HasCrCard
☒ IsActiveMember
☒ EstimatedSalary

Target (1)
☒ Exited

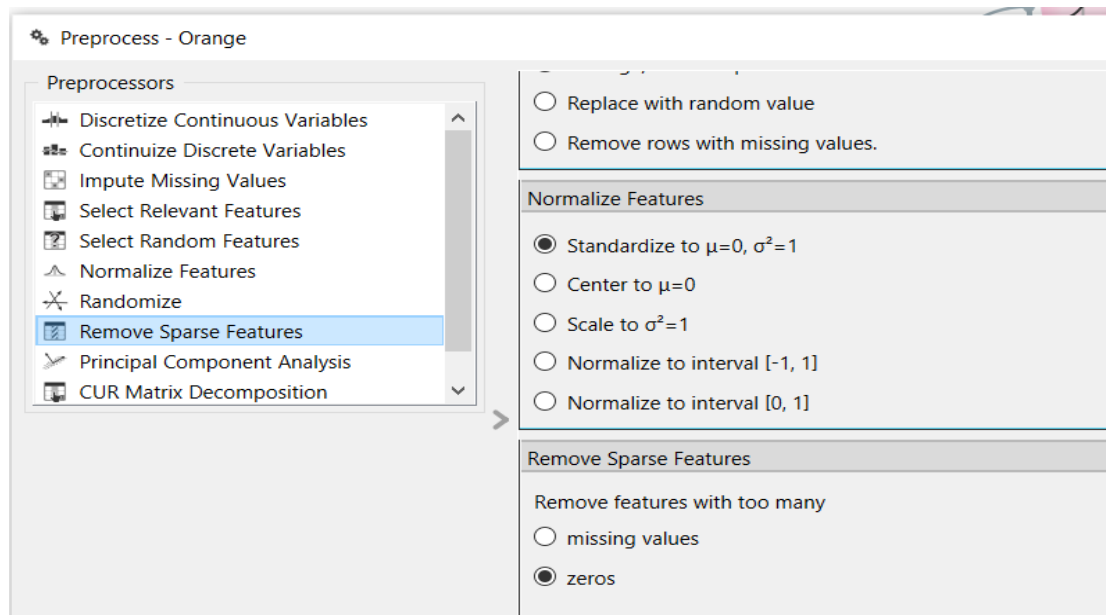
Metas

Reset ☐ Ignore new variables by default ☒ Send Automatically

10k 10k 10k

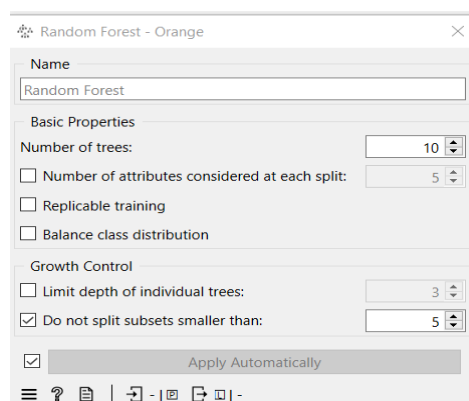
Step 3: Perform Data Preprocessing

Connect a **Preprocess** widget and enable **Min-Max Normalization**. This transformation scales all numerical attributes to a range between 0 and 1, ensuring consistent model training.



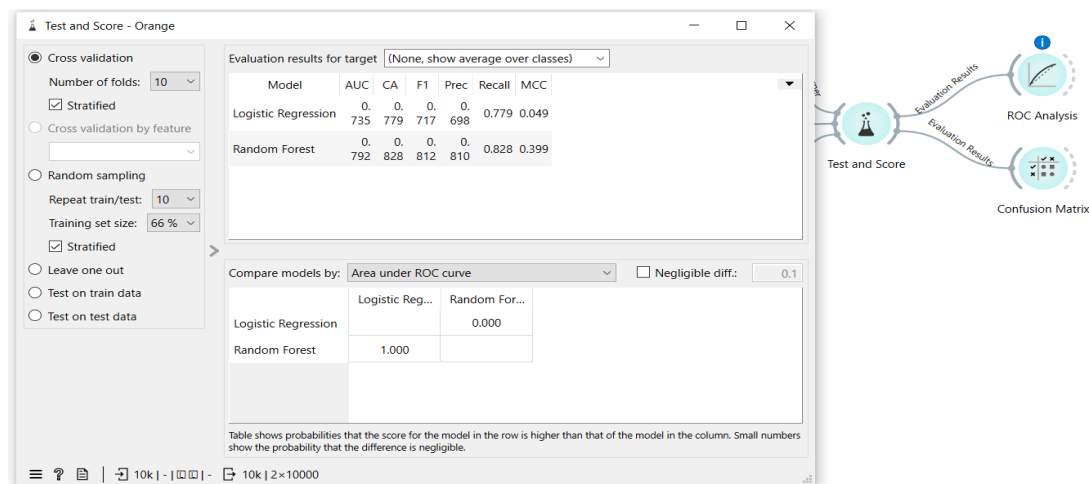
Step 4: Develop Classification Models

Link the preprocessed dataset to both **Logistic Regression** and **Random Forest** widgets through the Preprocessor output. These models will be trained to predict customer churn behavior.



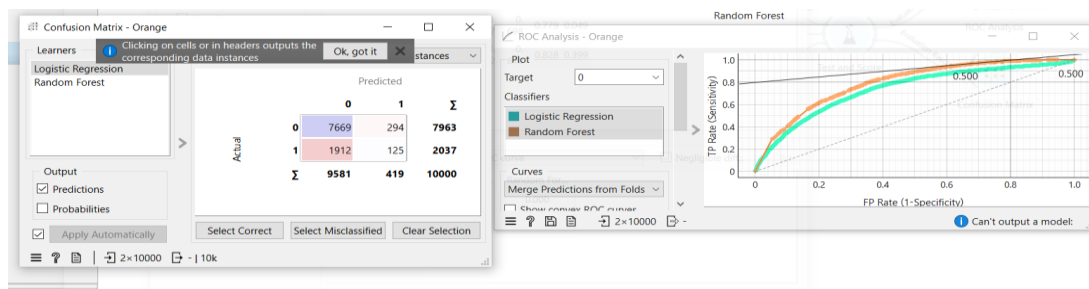
Step 5: Conduct Model Validation

Connect both classification models along with the processed dataset to the **Test and Score** widget. Configure the evaluation method to **10-Fold Cross Validation** and activate **Stratified Sampling** for balanced testing.



Step 6: Analyze Model Performance

Connect the Test and Score widget to both **ROC Analysis** and **Confusion Matrix** widgets. These visualizations provide insights into prediction accuracy, classification errors, and overall model effectiveness.

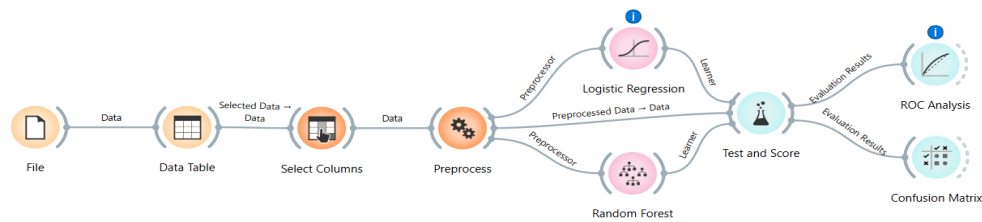


Results and Interpretation

This laboratory exercise demonstrates the complete process of transforming raw customer account information into a predictive churn analysis solution using the CRISP-DM methodology within Orange Data Mining.

The workflow follows the sequence:

Customer Data → Data Preparation → Model Training → Performance Evaluation → Business Insights



The comparative analysis showed that the **Random Forest** classifier achieved superior performance compared to **Logistic Regression** across multiple evaluation metrics. For churn prediction problems where class imbalance exists, metrics such as **AUC** and **F1-Score** provide a more meaningful assessment than overall accuracy.

The developed model can assist organizations in identifying customers with a high probability of churn, enabling timely retention campaigns and improving long-term customer loyalty.

Lab # 3

Healthcare Data Cleaning and Quality Enhancement

Real-World Problem

Healthcare organizations frequently collect large volumes of patient information from multiple sources. However, these datasets often contain data quality issues such as incomplete records, duplicated entries, inconsistent text formatting, and invalid numerical values. Such problems can negatively affect clinical reporting, operational planning, and analytical outcomes. To improve data reliability and support accurate healthcare analytics, a structured data cleansing process must be performed before analysis.

Dataset Description

This practical exercise utilizes a Healthcare Dataset containing detailed patient-level records.

Dataset Characteristics

- **Dataset Size:** 55,500 records and 15 attributes
- **Major Variables:** Patient Name, Age, Gender, Blood Type, Medical Condition, Billing Amount, Admission Type, and Medication
- **Data Quality Challenges:** Missing information, duplicate records, inconsistent text formatting, and unrealistic numerical values

The dataset serves as an appropriate case study for demonstrating healthcare data preparation and quality improvement techniques.

Tools and Technologies

RapidMiner Studio / Python (Pandas & NumPy)

RapidMiner Studio and Python libraries such as Pandas and NumPy are employed to perform data preprocessing, transformation, validation, and quality assessment tasks. These tools provide efficient methods for handling large healthcare datasets and automating cleaning procedures.

Practical Objectives

After completing this lab activity, students will be able to:

- 1. Detect and manage missing values within healthcare records.
- 2. Identify and remove duplicate patient entries.
- 3. Improve data consistency by standardizing text-based attributes.
- 4. Detect and eliminate invalid numerical observations.
- 5. Evaluate data quality using predefined performance indicators.
- 6. Export a cleaned dataset suitable for advanced analytics and machine learning applications.

Methodology and Implementation

Step 1: Initialize the Working Environment

Begin by importing the required Python libraries, including Pandas and NumPy, and load the healthcare dataset into the analytical environment for further processing.

Mount Google Drive

```
from google.colab import drive
drive.mount('/content/drive')

Mounted at /content/drive
```

Load Dataset

```
# IMPORT LIBRARIES AND LOAD DATASET
import pandas as pd
import numpy as np

# Local uploaded file
health_df = pd.read_csv("healthcare_dataset.csv")

print(f"Dataset Shape: {health_df.shape[0]} rows x {health_df.shape[1]} columns")

health_df.head()
```

Dataset Shape: 55500 rows x 15 columns

	Name	Age	Gender	Blood Type	Medical Condition	Date of Admission	Doctor	Hospital	Insurance Provider	Billing Amount	Room Number	Admission Type	Discharge Date	Medication	Test Results
0	Bobby Jackson	30	Male	B-	Cancer	2024-01-31	Matthew Smith	Sons and Miller	Blue Cross	10856.281306	328	Urgent	2024-02-02	Paracetamol	Normal
1	Leslie Terry	62	Male	A+	Obesity	2019-08-20	Samantha Davies	Kim Inc	Medicare	33643.327287	265	Emergency	2019-06-26	Ibuprofen	Inconclusive
2	Daniel Smith	76	Female	A-	Obesity	2022-09-22	Tiffany Mitchell	Cook PLC	Aetna	27955.096079	205	Emergency	2022-10-07	Aspirin	Normal
3	Andrew Watts	28	Female	O+	Diabetes	2020-11-18	Kevin Wells	Hernandez Rogers and Vang	Medicare	37909.782410	450	Elective	2020-12-18	Ibuprofen	Abnormal
4	Adrienne Bell	43	Female	AB+	Cancer	2022-09-19	Kathleen Hanna	White-White	Aetna	14238.317814	458	Urgent	2022-10-09	Penicillin	Abnormal

Next steps: [Generate code with health_df](#) [New interactive sheet](#)

Step 2: Perform Exploratory Data Assessment

Examine the structure of the dataset by reviewing the number of rows and columns. Analyze each attribute to identify missing values, inconsistencies, and other potential quality issues.

Initial Data Quality Audit

```
# INITIAL DATA QUALITY AUDIT

# Check missing values
missing_values = health_df.isnull().sum()

# Check duplicate records
duplicate_count = health_df.duplicated().sum()

# Identify negative billing amounts
negative_billing = (health_df["Billing Amount"] < 0).sum()

print("Missing Values:")
print(missing_values)

print(f"\nDuplicate Records: {duplicate_count}")
print(f"Negative Billing Records: {negative_billing}")
```

```
... Missing Values:
Name                0
Age                 0
Gender              0
Blood Type          0
Medical Condition   0
Date of Admission   0
Doctor              0
Hospital            0
Insurance Provider  0
Billing Amount       0
Room Number         0
Admission Type       0
Discharge Date       0
Medication           0
Test Results         0
dtype: int64

Duplicate Records: 534
Negative Billing Records: 108
```

Step 3: Eliminate Duplicate Entries

Inspect the dataset for repeated patient records and remove duplicate observations. This process ensures that each patient entry is represented only once within the dataset.

Remove Duplicate Records

```
# DUPLICATE RECORD CLEANING

initial_rows = len(health_df)

# Remove duplicates
health_df = health_df.drop_duplicates()

final_rows = len(health_df)

duplicates_removed = initial_rows - final_rows
duplicate_removal_pct = (duplicates_removed / initial_rows) * 100

print(f"Removed {duplicates_removed} duplicate records.")
print(f"Duplicate Reduction Rate: {duplicate_removal_pct:.2f}%")
```

```
... Removed 534 duplicate records.
Duplicate Reduction Rate: 0.96%
```

Step 4: Improve Data Consistency and Remove Noise

Standardize textual fields by applying consistent capitalization and formatting rules. In addition, detect and remove illogical numerical values, such as billing amounts that are less than or equal to zero.

Data Standardization & Noise Filtering

DATA STANDARDIZATION AND NOISE FILTERING

Standardize patient names
health_df["Name"] = health_df["Name"].str.title()

Convert negative billing amounts into positive values
health_df["Billing Amount"] = health_df["Billing Amount"].abs()

Remove extra spaces from text columns
text_columns = health_df.select_dtypes(include="object").columns

for col in text_columns:
 health_df[col] = health_df[col].str.strip()

print("Text standardization and noise correction completed successfully.")

health_df.head()

Text standardization and noise correction completed successfully.

	Name	Age	Gender	Blood Type	Medical Condition	Date of Admission	Doctor	Hospital	Insurance Provider	Billing Amount	Room Number	Admission Type	Discharge Date	Medication	Test Results
0	Bobby Jackson	30	Male	B-	Cancer	2024-01-31	Matthew Smith	Sons and Miller	Blue Cross	18556.261306	328	Urgent	2024-02-02	Paracetamol	Normal
1	Leslie Terry	62	Male	A+	Obesity	2019-08-20	Samantha Davies	Kim Inc	Medicare	33643.327287	265	Emergency	2019-08-26	Ibuprofen	Inconclusive
2	Danny Smith	76	Female	A-	Obesity	2022-09-22	Tiffany Mitchell	Cook PLC	Aetna	27955.096079	205	Emergency	2022-10-07	Aspirin	Normal
3	Andrew Watts	28	Female	O+	Diabetes	2020-11-16	Kevin Wells	Hernandez Rogers and Vang,	Medicare	37909.762410	450	Elective	2020-12-18	Ibuprofen	Abnormal
4	Adrienne Bell	43	Female	AB+	Cancer	2022-09-19	Kathleen Hanna	White-White	Aetna	14238.317814	458	Urgent	2022-10-09	Penicillin	Abnormal

Next steps:

Generate code with health_df

New interactive sheet

Step 5: Assess Data Quality Metrics

Calculate key performance indicators to evaluate the effectiveness of the cleaning process. Metrics may include:

- Percentage of Missing Values Resolved
- Duplicate Record Reduction Rate
- Overall Data Quality Score

These indicators help quantify improvements made during preprocessing.

Calculate Data Quality KPIs

```
# DATA QUALITY KPI CALCULATION

# Remaining missing values
remaining_missing = health_df.isnull().sum().sum()

# Missing Value Reduction
if remaining_missing == 0:
    missing_reduction_pct = 100
else:
    missing_reduction_pct = (
        (health_df.size - remaining_missing) / health_df.size
    ) * 100

# Remaining invalid billing records
remaining_noise = (health_df["Billing Amount"] <= 0).sum()

# Data Quality Score
data_quality_score = (
    (len(health_df) - remaining_noise) / len(health_df)
) * 100

# KPI Table
kpi_table = pd.DataFrame({
    "KPI": [
        "Missing Value Reduction %",
        "Duplicate Removal %",
        "Data Quality Score"
    ],
    "Value": [
        f"{missing_reduction_pct:.2f}%",
        f"{duplicate_removal_pct:.2f}%",
        f"{data_quality_score:.2f}%"
    ]
})

kpi_table
```

	KPI	Value
0	Missing Value Reduction %	100.00%
1	Duplicate Removal %	0.96%
2	Data Quality Score	100.00%

Next steps: [Generate code with kpi_table](#) [New interactive sheet](#)

Step 6: Store the Refined Dataset

Export the cleaned and validated dataset into a new file, such as **cleaned_healthcare_data.csv**, to support future reporting, analysis, and predictive modeling tasks.

Export Cleaned Dataset

```
# EXPORT CLEANED DATASET

output_file = "cleaned_healthcare_dataset.csv"

health_df.to_csv(output_file, index=False)

print(f"Cleaned dataset exported successfully as '{output_file}'")

Cleaned dataset exported successfully as 'cleaned_healthcare_dataset.csv'
```

Download Cleaned File

```
# DOWNLOAD CLEANED DATASET

from google.colab import files

files.download("cleaned_healthcare_dataset.csv")
```

 Start coding or [generate](#) with AI.

Results and Key Findings

This laboratory exercise highlights the importance of data preprocessing within healthcare analytics. Through systematic cleaning and validation procedures, raw patient records are transformed into a structured and dependable dataset suitable for analytical applications.

The workflow follows the progression:

Raw Healthcare Data → Data Cleaning → Quality Validation → Analysis-Ready Dataset

Major Achievements

Improved Data Integrity

Duplicate records and inconsistent entries were successfully removed, reducing data redundancy and enhancing the reliability of healthcare reporting.

Enhanced Logical Accuracy

Invalid observations, including unrealistic billing values, were identified and corrected, ensuring that financial information accurately reflects healthcare operations.

Consistent Data Formatting

Text-based attributes such as patient names, medical conditions, and admission types were standardized to improve classification accuracy and database management.

Measurable Data Quality Improvements

Quantitative indicators, including Data Quality Score and Missing Value Reduction Percentage, were generated to objectively assess dataset readiness for advanced analytics and machine learning applications.

Conclusion

The completed workflow demonstrates how effective data cleaning techniques can significantly improve the reliability, consistency, and usability of healthcare datasets. By addressing common data quality issues, organizations can establish a strong foundation for accurate medical analysis, informed decision-making, and predictive healthcare modeling.

Lab # 04

Data Preprocessing

Real-World Problem

Financial institutions must carefully evaluate the risk associated with loan applicants before approving credit requests. However, raw loan datasets often contain incomplete records, varying measurement scales, and continuous variables that are not immediately suitable for predictive analysis. To ensure accurate assessment and modeling, the data must undergo a series of preprocessing steps. This lab focuses on preparing loan application data using RapidMiner Studio to create a reliable and structured dataset for future analytical tasks.

Dataset Overview

The study utilizes the **loan_dataset_20000.csv** dataset, which contains information from 20,000 loan applications.

Key Variables Included

- Age
- Annual Income
- Monthly Income
- Debt-to-Income Ratio
- Credit Score
- Loan Amount
- Interest Rate
- Loan Term
- Installment Amount

Following the data cleaning process, records containing incomplete information were addressed, resulting in **8,231 valid observations** available for further processing and analysis.

Analytical Platform

RapidMiner Studio

RapidMiner Studio is a workflow-based data analytics environment that enables users to design preprocessing and analytical pipelines through graphical operators. The

platform supports data transformation, cleaning, normalization, and statistical exploration without requiring programming expertise.

Practical Objectives

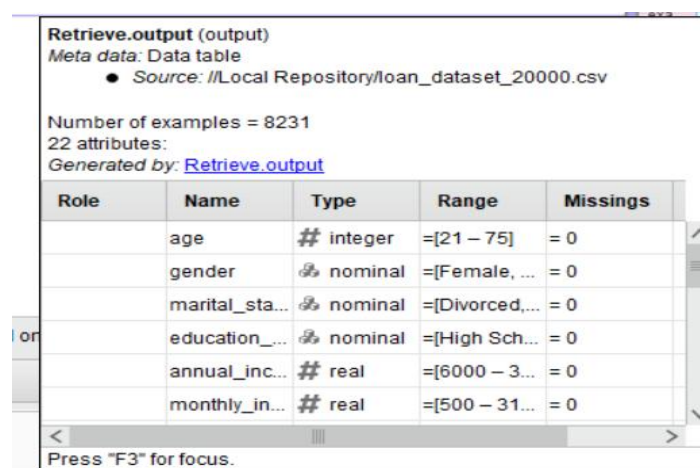
By completing this lab, students will be able to:

1. Import a CSV dataset using the Retrieve operator.
2. Resolve missing data issues through the Replace Missing Values operator.
3. Standardize numerical attributes using Z-Score normalization.
4. Reduce the impact of extreme values through Z-Score Peak Transformation.
5. Convert continuous variables into categorized intervals.
6. Generate descriptive statistical reports using the Statistics operator.
7. Evaluate preprocessing effectiveness using performance indicators such as Variance Reduction and Distribution Improvement.

Experimental Procedure

Step 1: Import the Loan Dataset

Begin by using the **Retrieve** operator to access the *loan_dataset_20000.csv* file. Connect the dataset to subsequent operators to establish the preprocessing workflow.



Retrieve.output (output)
 Meta data: Data table
 ● Source: //Local Repository/loan_dataset_20000.csv

Number of examples = 8231
 22 attributes:
 Generated by: [Retrieve.output](#)

Role	Name	Type	Range	Missings
	age	# integer	=[21 – 75]	= 0
	gender	nominal	=[Female, ...]	= 0
	marital_sta...	nominal	=[Divorced, ...]	= 0
	education_...	nominal	=[High Sch...]	= 0
	annual_inc...	# real	=[6000 – 3...]	= 0
	monthly_in...	# real	=[500 – 31...]	= 0

Press "F3" for focus.

Step 2: Address Incomplete Records

Attach a **Replace Missing Values** operator to the workflow. This step handles missing entries across all attributes and ensures that only valid records remain available for analysis.

Replace Missing Values

Replace Missing Values.original (original)
 Meta data: Data Table
 ● Source: //Local Repository/loan_dataset_20000.csv

Number of examples = 8231
 22 attributes:
 Generated by: [Replace Missing Values.original](#) ← [Retrieve.output](#)

Role	Name	Type	Range	Missings
	age	# integer	=[21 – 75]	= 0
	gender	polyno...	=[Female, ...]	= 0
	marital_sta...	polyno...	=[Divorced, ...]	= 0
	education_...	polyno...	=[High Sch...]	= 0
	annual_inc...	# real	=[6000 – 3...]	= 0
	monthly_in...	# real	=[500 – 31...]	= 0

Step 3: Standardize Numerical Features

Use the **Normalize** operator and select the **Z-Score** method. This transformation standardizes numerical variables by centering them around a mean of zero and scaling them according to their standard deviation.

Normalize

Normalize.original (original)
 Meta data: Data Table
 ● Source: //Local Repository/loan_dataset_20000.csv

Number of examples = 8231
 22 attributes:
 Generated by: [Normalize.original](#) ← [Replace Missing Values.original](#)
 ← [Retrieve.output](#)

Role	Name	Type	Range	Missings
	age	# integer	=[21 – 75]	= 0
	gender	polyno...	=[Female, ...]	= 0
	marital_sta...	polyno...	=[Divorced, ...]	= 0
	education_...	polyno...	=[High Sch...]	= 0
	annual_inc...	# real	=[6000 – 3...]	= 0
	monthly_in...	# real	=[500 – 31...]	= 0

Press "F3" for focus.

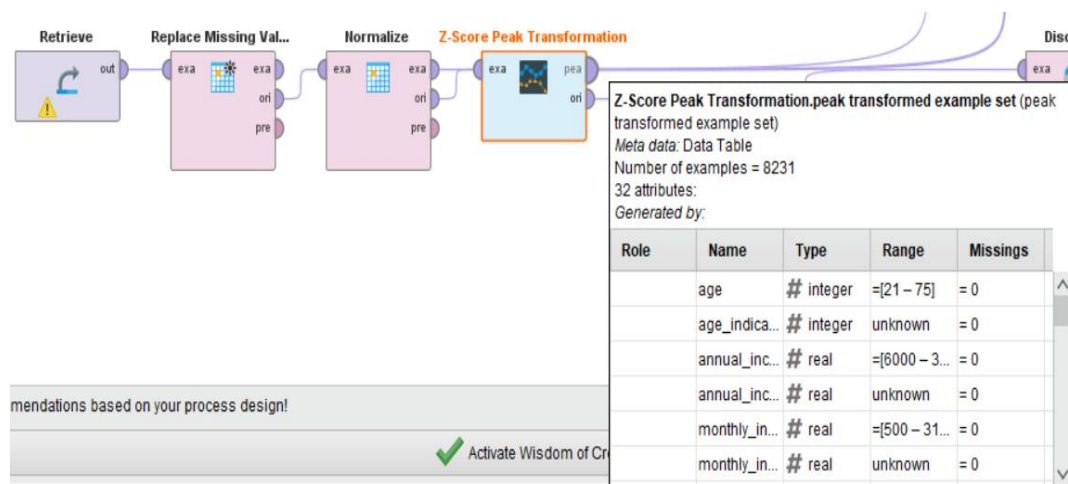
Open in Turbo Prep Auto Model

Filter (8,231 / 8,231 examples): all

Row No.	age	annual_inco...	monthly_inc...	debt_to_inc...	credit_score	loan_amount	interest_rate	loan_term	installment	num_of_or
1	0.068	-0.595	-0.595	0.546	1.438	-1.350	-1.178	1.488	-1.391	-1.337
2	-0.817	-1.085	-1.085	0.833	1.870	-0.032	-1.803	-0.672	0.043	0.870
3	0.952	-0.624	-0.624	0.795	-0.230	0.156	1.126	1.488	-0.222	-1.778
4	-1.259	0.409	0.409	-0.920	1.366	-0.290	0.123	1.488	-0.617	-0.895
5	-0.564	0.661	0.661	-0.987	0.647	-1.715	-1.007	-0.672	-1.611	0.429
6	-0.311	0.337	0.337	1.896	1.683	-0.058	-0.251	-0.672	0.114	-0.454
7	-1.069	-0.223	-0.223	-0.106	0.618	1.123	1.110	-0.672	1.490	-0.895
8	-0.374	0.515	0.515	0.316	-1.453	0.105	1.955	1.488	-0.196	-0.454
9	1.015	0.912	0.912	-0.738	-3.438	-0.741	0.769	-0.672	-0.557	-0.454
10	0.131	-0.108	-0.108	-1.054	1.064	0.490	-0.259	-0.672	0.684	0.429
11	0.636	-0.828	-0.828	-0.355	-0.863	0.340	0.574	-0.672	0.593	1.312
12	-1.638	-1.038	-1.038	-0.527	0.431	-0.643	0.509	-0.672	-0.463	0.429
13	-0.438	-0.911	-0.911	2.375	0.374	0.398	0.444	-0.672	0.645	-0.895
14	-1.006	-1.221	-1.221	0.431	0.187	-1.100	-0.068	-0.672	-0.966	1.753
15	1.015	0.166	0.166	0.172	-0.992	-1.157	-0.775	1.488	-1.254	-1.778
16	-0.185	-0.535	-0.535	-0.431	-1.812	-1.043	1.049	-0.672	-0.875	-0.895
17	0.889	-0.319	-0.319	-0.833	0.086	-0.529	-0.304	-0.672	-0.378	0.870
18	-0.753	0.141	0.141	0.392	1.827	0.928	-2.067	-0.672	0.962	-0.012

Step 4: Treat Extreme Observations

Apply the **Z-Score Peak Transformation** operator to identify and minimize the influence of unusual or extreme values that may distort statistical analysis and model performance.



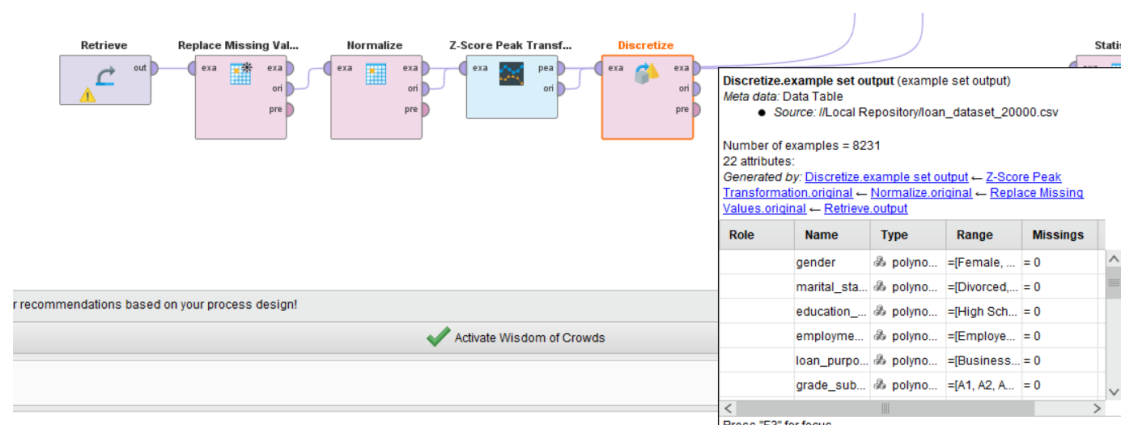
Open in Turbo Prep Auto Model

Filter (8,231 / 8,231 examples): all

Row No.	age	age_indicator	age_peaked	annual_inco...	annual_inco...	annual_inco...	monthly_inc...	monthly_inc...	monthly_inc...	debt_to_in
1	49	?	?	26181.800	?	?	2181.820	?	?	0.234
2	35	?	?	11873.840	?	?	989.490	?	?	0.264
3	63	?	?	25326.440	?	?	2110.540	?	?	0.260
4	28	?	?	55559.800	?	?	4629.980	?	?	0.081
5	39	?	?	62922.050	?	?	5243.500	?	?	0.074
6	43	?	?	53439.890	?	?	4453.320	?	?	0.375
7	31	?	?	37067.580	?	?	3088.960	?	?	0.166
8	42	?	?	58633.230	?	?	4886.100	?	?	0.210
9	64	?	?	70246.140	?	?	5853.840	?	?	0.100
10	50	?	?	40422.150	?	?	3368.510	?	?	0.067
11	58	0	?	19364.050	0	?	1613.670	0	?	0.140
12	22	0	?	13220.540	0	?	1101.710	0	?	0.122
13	41	0	?	16936.710	0	?	1411.390	0	?	0.425
14	32	0	?	7884.140	0	?	657.010	0	?	0.222
15	64	0	?	48453.690	0	?	4037.810	0	?	0.195
16	45	0	?	27944.460	0	?	2328.700	0	?	0.132
17	62	0	?	34248.840	0	?	2854.070	0	?	0.090

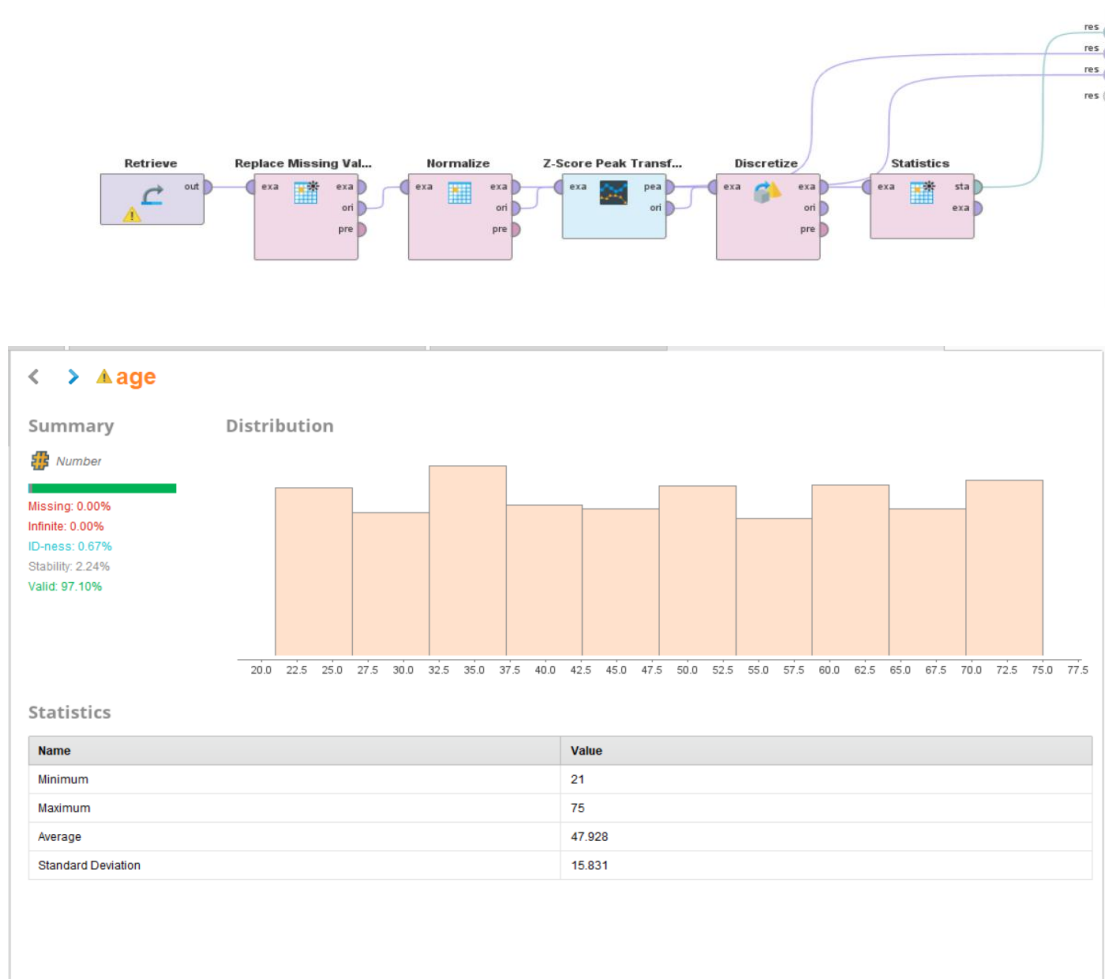
Step 5: Convert Continuous Data into Categories

Utilize the **Discretize** operator to transform numerical attributes into predefined intervals or bins. This conversion improves compatibility with rule-based and classification techniques that operate on categorical inputs.



Step 6: Perform Statistical Analysis

Connect the processed dataset to a **Statistics** operator to generate descriptive measures for each attribute, including minimum values, maximum values, averages, and standard deviations.



Results and Discussion

This exercise demonstrates how a raw loan application dataset can be systematically refined through a sequence of preprocessing operations in RapidMiner Studio.

The workflow follows the progression:

Raw Loan Data → Data Cleaning → Feature Transformation → Statistical Validation → Analysis-Ready Dataset

Performance Indicators

KPI	Result
Records Available After Processing	8,231 (from 20,000)
Minimum Age	21
Maximum Age	75
Average Age	47.928
Age Standard Deviation	15.831

KPI	Result
Remaining Missing Values	0.00%
Valid Data Rate	97.10%

Key Observations

- Missing and incomplete records were successfully addressed during preprocessing.
- Z-Score normalization standardized all numerical variables, reducing scale-related inconsistencies.
- Z-Score Peak Transformation minimized the effect of extreme observations and improved data distribution.
- Discretization converted continuous attributes into meaningful categories suitable for downstream analytical models.
- The resulting dataset is fully prepared for predictive modeling and risk assessment activities in future experiments.

Conclusion

The preprocessing workflow significantly improved the quality and consistency of the loan application dataset. Through data cleaning, normalization, outlier treatment, and discretization, the dataset was transformed into a structured format that supports reliable statistical analysis and machine learning applications. The prepared data provides a strong foundation for developing accurate loan risk prediction models.

Lab # 05

Market Basket Analysis

Real-World Problem

Retail stores generate large volumes of transaction data every day. While this data contains valuable information about customer purchasing behavior, it often remains underutilized. Understanding which products are commonly purchased together can help retailers improve product placement, design promotional bundles, and create effective cross-selling campaigns. Through market basket analysis, supermarkets can uncover purchasing patterns that contribute to higher sales and a better shopping experience for customers.

Dataset Overview

This analysis uses the **Groceries Dataset**, which contains customer shopping records collected from a retail environment.

Dataset Characteristics

- **Attributes:** Member_number, Date, and ItemDescription
- **Transaction Definition:** Products purchased by the same customer on the same date are grouped into a single transaction basket.
- **Dataset Size:** Approximately 14,963 unique shopping transactions

The dataset is suitable for association rule mining and market basket analysis, allowing the discovery of relationships among purchased products.

Analytical Environment

Python-Based Analysis

The practical implementation is carried out using Python and its data analysis libraries.

- **Pandas:** Used for data preparation, cleaning, and transaction grouping.
- **Association Rule Metrics:** Calculated programmatically to measure relationships between products.
- **Transaction Analysis:** Applied to identify meaningful item combinations and customer purchasing patterns.

Practical Objectives

- **Frequent Itemset Discovery:** Determine products that frequently occur within customer baskets.
- **Association Analysis:** Measure relationships between products using Support, Confidence, and Lift.
- **Rule Verification:** Validate association metrics through both manual and automated calculations.
- **Business Evaluation:** Identify valuable cross-selling opportunities using predefined KPI thresholds.

Methodology and Implementation

Step 1: Transaction Creation and Data Structuring

The dataset is first organized by grouping records according to **Member Number** and **Purchase Date**. This process combines individual item purchases into complete shopping baskets, creating a transaction-based structure suitable for association analysis.

Load Grocery Data and Build Transactions

```
# Load Grocery Data and Build Transactions

import pandas as pd
import numpy as np
from itertools import combinations
from collections import Counter

grocery_df = pd.read_csv("Groceries_dataset.csv")

grocery_df["Transaction_ID"] = (
    grocery_df["Member_number"].astype(str)
    + "_"
    + grocery_df["Date"].astype(str)
)

transactions = (
    grocery_df.groupby("Transaction_ID")["itemDescription"]
    .apply(list)
    .tolist()
)

total_transactions = len(transactions)

print(f"Total Transactions: {total_transactions}")
print(transactions[0])

... Total Transactions: 14963
['sausage', 'whole milk', 'semi-finished bread', 'yogurt']
```

Step 2: Validation of Association Metrics

A sample product pair is selected to manually verify the calculations of Support, Confidence, and Lift.

- **Support** measures the frequency of occurrence of a product combination.
- **Confidence** indicates the probability of purchasing one product when another product is already present.
- **Lift** evaluates the strength of the relationship compared to random purchasing behavior.

This step ensures the correctness of the analytical methodology before applying it to the full dataset.

✓ Calculate Support, Confidence and Lift

```
[2]
✓ Os
# Calculate Support, Confidence and Lift

def calculate_metrics(item_a, item_b, transactions):

    count_a = sum(1 for t in transactions if item_a in t)
    count_b = sum(1 for t in transactions if item_b in t)
    count_ab = sum(
        1 for t in transactions
        if item_a in t and item_b in t
    )

    support_a = count_a / total_transactions
    support_b = count_b / total_transactions
    support_ab = count_ab / total_transactions

    confidence = (
        support_ab / support_a
        if support_a > 0 else 0
    )

    lift = (
        confidence / support_b
        if support_b > 0 else 0
    )

    return {
        "Support": support_ab,
        "Confidence": confidence,
        "Lift": lift
    }

# Example calculation
metrics = calculate_metrics(
    "whole milk",
    "yogurt",
    transactions
)

print("Metrics for {whole milk → yogurt}")

for metric, value in metrics.items():
    print(f"{metric}: {value:.4f}")
```

```
Metrics for {whole milk → yogurt}
Support: 0.0112
Confidence: 0.0707
Lift: 0.8229
```


Step 3: Association Rule Extraction

All possible product combinations are evaluated across the transaction dataset. Association rules are generated in both directions, allowing the identification of relationships such as Product A $\rightarrow$ Product B and Product B $\rightarrow$ Product A.

Each rule is evaluated using:

- Support
- Confidence
- Lift

The resulting rules are ranked to highlight the strongest product associations.

Generate Frequent Itemsets and Association Rules

```
# Generate Frequent Itemsets and Association Rules

# Calculate item support
all_items = [
    item
    for basket in transactions
    for item in basket
]

item_counts = Counter(all_items)

item_support = {
    item: count / total_transactions
    for item, count in item_counts.items()
}

# Calculate pair support
pair_counts = Counter()

for basket in transactions:
    unique_items = sorted(set(basket))

    if len(unique_items) > 1:
        for pair in combinations(unique_items, 2):
            pair_counts[pair] += 1

# Generate association rules
rules = []

for pair, count in pair_counts.items():
    item_a, item_b = pair

    support_ab = count / total_transactions

    conf_ab = support_ab / item_support[item_a]
    lift_ab = conf_ab / item_support[item_b]

    conf_ba = support_ab / item_support[item_b]
    lift_ba = conf_ba / item_support[item_a]
```

```

    }
    0s
        rules.append([
            item_a,
            item_b,
            support_ab,
            conf_ab,
            lift_ab
        ])

        rules.append([
            item_b,
            item_a,
            support_ab,
            conf_ba,
            lift_ba
        ])

    # Create rules dataframe
    rules_df = pd.DataFrame(
        rules,
        columns=[
            "Antecedent",
            "Consequent",
            "Support",
            "Confidence",
            "Lift"
        ]
    )

    # Sort rules by lift
    rules_df = (
        rules_df
        .sort_values("Lift", ascending=False)
        .reset_index(drop=True)
    )

    print("Top 10 High Lift Rules")

    rules_df.head(10)

```

*** Top 10 High Lift Rules

	Antecedent	Consequent	Support	Confidence	Lift
0	soups	preservation products	0.000067	0.020833	311.729167
1	preservation products	soups	0.000067	1.000000	311.729167
2	artif. sweetener	baby cosmetics	0.000067	0.034483	171.988506
3	baby cosmetics	artif. sweetener	0.000067	0.333333	171.988506
4	kitchen utensil	pasta	0.000067	1.000000	123.661157
5	pasta	kitchen utensil	0.000067	0.008264	123.661157
6	sparkling wine	rubbing alcohol	0.000067	0.021739	65.056522
7	rubbing alcohol	sparkling wine	0.000067	0.200000	65.056522
8	prosecco	honey	0.000067	0.052632	60.578947
9	honey	prosecco	0.000067	0.076923	60.578947

Step 4: Selection of High-Value Rules

The generated rules are filtered using predefined thresholds to identify the most meaningful relationships.

Selection Criteria:

- **Lift > 1**
- **Confidence > 0.05**

Rules meeting these conditions represent potentially valuable product combinations that can support retail decision-making and cross-selling initiatives.

Identify Cross-Selling Opportunities

```

# Identify Cross-Selling Opportunities

significant_rules = rules_df[
    (rules_df["Lift"] > 1)
    &
    (rules_df["Confidence"] > 0.05)
]

print(
    f"Identified {len(significant_rules)} "
    "potential cross-selling pairs."
)

significant_rules.head(15)

```

... Identified 1005 potential cross-selling pairs.

	Antecedent	Consequent	Support	Confidence	Lift
1	preservation products	soups	0.000067	1.000000	311.729167
3	baby cosmetics	artif. sweetener	0.000067	0.333333	171.988506
4	kitchen utensil	pasta	0.000067	1.000000	123.661157
7	rubbing alcohol	sparkling wine	0.000067	0.200000	65.056522
8	prosecco	honey	0.000067	0.052632	60.578947
9	honey	prosecco	0.000067	0.076923	60.578947
10	whisky	rum	0.000067	0.125000	58.449219
12	cookware	spices	0.000134	0.117647	44.008824
14	honey	artif. sweetener	0.000067	0.076923	39.689655
17	toilet cleaner	roll products	0.000067	0.200000	36.495122
18	decalcifier	curd cheese	0.000067	0.111111	36.142512
20	decalcifier	soups	0.000067	0.111111	34.636574
23	cooking chocolate	light bulbs	0.000067	0.066667	34.397701

Step 5: Business Interpretation and Recommendations

The final set of association rules is reviewed to identify actionable opportunities. High-performing product pairs can be used to support:

- Product placement optimization
- Promotional bundle creation
- Cross-selling campaigns
- Personalized marketing strategies
- Store layout improvements

These insights help retailers encourage complementary purchases and increase transaction value.

Export Association Rules

```

js  # Export Association Rules

    significant_rules.to_csv(
        "cross_sell_rules.csv",
        index=False
    )

    print("Association rules exported successfully.")

```

... Association rules exported successfully.

Download Results File (Optional)

```

js  # Download Results File (Optional)

    from google.colab import files

    files.download("cross_sell_rules.csv")

```

Results and Key Findings

This exercise demonstrates how transactional retail data can be transformed into meaningful business intelligence through association rule mining techniques.

The analytical workflow follows:

Transaction Data → Basket Formation → Association Analysis → Rule Filtering → Business Insights

Major Outcomes

Hidden Purchasing Patterns Identified

The analysis successfully uncovered relationships among products that frequently appear together in customer transactions, revealing buying behaviors that may not be immediately visible through traditional reporting.

Improved Cross-Selling Opportunities

Association rules provided clear evidence of product combinations that can be leveraged to increase sales through targeted recommendations and promotional strategies.

Reliable Analytical Framework

Manual verification of Support, Confidence, and Lift calculations confirmed the accuracy of the automated analysis process, ensuring trustworthy results.

Strategic Retail Insights

The discovered item associations can be applied to improve store organization, promotional planning, and customer engagement strategies, ultimately supporting revenue growth and customer satisfaction.

Conclusion

Market basket analysis provides a powerful method for understanding customer purchasing behavior. By converting raw transaction records into association rules, retailers can identify valuable product relationships and develop data-driven merchandising strategies. The resulting insights support effective cross-selling, optimized product placement, and improved business performance.

Lab # 06

Apriori Algorithm

Real-World Problem

Retail businesses process a large number of transactions every day, making it difficult to manually identify customer purchasing trends. Hidden relationships among product categories can provide valuable insights for marketing and sales strategies. By applying the Apriori algorithm, retailers can uncover frequently purchased product categories and use this knowledge to improve cross-selling, promotional campaigns, and customer engagement.

Dataset Overview

This analysis uses the Retail Sales Dataset, which contains customer transaction records across multiple product categories.

- Attributes: Transaction ID, Customer ID, Gender, Age, Product Category, Quantity, Price per Unit, and Total Amount.
- Structure: Each record represents a customer purchase transaction. Transactions are grouped using Customer ID to create customer purchase baskets.
- Product Categories: Beauty, Clothing, and Electronics.
- Purpose: To discover recurring purchase patterns and associations among product categories using market basket analysis.

Software Environment

- Python: Used for data preprocessing, transformation, and analysis through the Pandas library.
- mlxtend Library: Provides the Apriori algorithm and association rule generation functions.
- Google Colab: Used as the execution environment for running the analysis.

Practical Objectives

- Basket Construction: Transform transaction records into a binary customer-product matrix.
- Frequent Itemset Discovery: Identify product categories that commonly occur in customer purchase histories.
- Association Rule Mining: Generate rules and evaluate them using Support, Confidence, and Lift.
- Threshold Evaluation: Examine how different minimum support values affect the number of discovered rules.

Procedure & Steps

Step 1: Environment Setup and Data Preparation

Import the required Python libraries and load the retail sales dataset. Convert transaction records into a customer-product matrix and encode purchases as binary values (1 = purchased, 0 = not purchased) to prepare the data for Apriori analysis.

Load required libraries and retail dataset

```
# Load required libraries and retail dataset

import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules

retail_data = pd.read_csv("retail_sales_dataset.csv")

print("Dataset Shape:", retail_data.shape)
retail_data.head()
```

Dataset Shape: (1000, 9)
 /usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is return datetime.datetime.utcnow().replace(tzinfo=utc)
 /usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is return datetime.datetime.utcnow().replace(tzinfo=utc)

	Transaction ID	Date	Customer ID	Gender	Age	Product Category	Quantity	Price per Unit	Total Amount
0	1	2023-11-24	CUST001	Male	34	Beauty	3	50	150
1	2	2023-02-27	CUST002	Female	26	Clothing	2	500	1000
2	3	2023-01-13	CUST003	Male	50	Electronics	1	30	30
3	4	2023-05-21	CUST004	Male	37	Clothing	1	500	500
4	5	2023-05-06	CUST005	Male	30	Beauty	2	50	100

/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is

Step 2: Frequent Itemset Generation

Apply the Apriori algorithm with a selected minimum support threshold to identify frequently occurring product categories. Sort the resulting itemsets by support to highlight the most common purchasing patterns.

Convert transactions into customer-category purchase matrix

```
# Convert transactions into customer-category purchase matrix

transaction_matrix = pd.crosstab(
    retail_data['Customer ID'],
    retail_data['Product Category']
)

binary_matrix = transaction_matrix.astype(bool)

print("Binary Matrix Shape:", binary_matrix.shape)
binary_matrix.head()
```

/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: da return datetime.datetime.utcnow().replace(tzinfo=utc)
 /usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: da return datetime.datetime.utcnow().replace(tzinfo=utc)

Product Category Beauty Clothing Electronics



Customer ID

Customer ID	Beauty	Clothing	Electronics
CUST001	True	False	False
CUST002	False	True	False
CUST003	False	False	True
CUST004	False	True	False
CUST005	True	False	False

/usr/local/lib/python3.12/dist-packages/jupyter_client/s

Step 3: Association Rule Generation

Generate association rules from the frequent itemsets using Lift as the primary evaluation metric. Analyze the relationships between antecedents and consequents to identify meaningful product associations.

Find frequently purchased product categories

▶ # Find frequently purchased product categories

```
frequent_patterns = apriori(
    binary_matrix,
    min_support=0.01,
    use_colnames=True
)

frequent_patterns = frequent_patterns.sort_values(
    by='support',
    ascending=False
)

print("Frequent Itemsets Found:", len(frequent_patterns))
frequent_patterns.head(10)
```

```
... /usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: Deprec
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: Deprec
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: Deprec
```

return datetime.utcnow().replace(tzinfo=utc)

support itemsets



1	0.351	(Clothing)
2	0.342	(Electronics)
0	0.307	(Beauty)

```
/usr/local/lib/python3.12/dist-packages/jup
return datetime.utcnow().replace(tzinfo=utc)
```


Step 4: Support Threshold Analysis

Experiment with multiple support values (0.005, 0.01, 0.02, and 0.05) and compare the number of rules generated at each threshold. This helps evaluate how parameter selection influences the discovery process.

Discover associations among product categories

```
# Discover associations among product categories

rules = association_rules(
    frequent_patterns,
    metric='lift',
    min_threshold=1.0
)

rules_summary = rules[
    ['antecedents',
     'consequents',
     'support',
     'confidence',
     'lift']]

rules_summary = rules_summary.sort_values(
    by='lift',
    ascending=False
)

print("Total Rules Generated:", len(rules_summary))
rules_summary.head(10)
```

```
... /usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationW
    return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationW
    return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationW
```

Step 5: Business Insights and Conclusion

Interpret the discovered associations to identify potential cross-selling opportunities, product recommendations, and promotional strategies. Assess the effectiveness of different support thresholds for generating useful business insights.

Compare rule generation at different support levels

```
# Compare rule generation at different support levels

support_values = [0.005, 0.01, 0.02, 0.05]

rule_counts = []

for s in support_values:

    itemsets = apriori(
        binary_matrix,
        min_support=s,
        use_colnames=True
    )

    if not itemsets.empty:

        generated_rules = association_rules(
            itemsets,
            metric='lift',
            min_threshold=1.0
        )

        rule_counts.append(len(generated_rules))

    else:
        rule_counts.append(0)

support_analysis = pd.DataFrame({
    'Support_Level': support_values,
    'Rules_Generated': rule_counts
})

support_analysis
```

	Support_Level	Rules_Generated
0	0.005	0
1	0.010	0
2	0.020	0
3	0.050	0

/usr/local/lib/python3.12/dist-packages/jupy

Outcome of the Lab

This experiment demonstrates how association rule mining can be used to extract valuable knowledge from retail transaction data.

- Automated Pattern Identification: Enabled efficient discovery of product relationships without manual analysis.

- Association Analysis: Revealed purchasing trends across Beauty, Clothing, and Electronics categories.
- Parameter Sensitivity: Showed that support threshold selection significantly affects the quantity and quality of generated rules.
- Business Value: Provided actionable insights for targeted marketing, personalized promotions, and improved retail decision-making.

Lab # 07

FP-Growth

Real-World Problem

As retail transaction datasets grow in scale, traditional algorithms like Apriori become computationally expensive due to repeated database scans and candidate generation. FP-Growth addresses this by compressing the entire transaction database into a single FP-Tree structure, enabling faster and more scalable frequent pattern mining without sacrificing accuracy.

Dataset Overview

The analysis is conducted on the New Retail Dataset, which records individual customer purchases across product types.

- **Attributes:** Transaction_ID, Product_Type, and Total_Purchases.
 - **Data Structure:** Each record represents a single product purchase. Transactions are formed by grouping products purchased under the same Transaction_ID.
 - **Dataset Size:** A transaction matrix consisting of 294,461 transactions and 33 distinct product categories.
-

Analytical Tools and Technologies

Python and MLxtend Framework

The implementation is performed using Python along with specialized libraries for transaction analysis and association rule mining.

- **Pandas:** Used for data cleaning, transformation, aggregation, and one-hot encoding.
 - **MLxtend:** Provides implementations of Apriori, FP-Growth, and Association Rule Mining algorithms.
 - **Time Module:** Used to measure algorithm execution times for performance comparison.
-

Practical Objectives

- **FP-Tree Construction:** Build a compressed FP-Tree from the transaction matrix for efficient pattern mining.
- **Performance Comparison:** Measure and compare execution time between FP-Growth and Apriori at the same support threshold.
- **Association Rule Mining:** Extract rules from FP-Growth results using Lift as the primary metric.
- **KPIs:** Evaluate both algorithms on Execution Time, Rule Count, and Efficiency Gain.

Experimental Methodology

Step 1: Configure the Working Environment

Begin by installing the required libraries and importing essential modules, including Pandas, Time, Apriori, FP-Growth, and Association Rules. These components provide the functionality necessary for transaction analysis and algorithm evaluation.

Environment Setup and Library Import

```

# Environment Setup and Library Import

!pip install mlxtend -q

import pandas as pd
import numpy as np
import time

from mlxtend.frequent_patterns import (
    apriori,
    fpgrowth,
    association_rules
)

```

... /usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware :
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware :
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware :
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware :
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware :
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware :
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware :
return datetime.utcnow().replace(tzinfo=utc)

Step 2: Prepare Transaction Data

Load the retail dataset and remove records containing missing values in critical fields. Organize the data by Transaction_ID and Product_Type, then transform it into a transaction matrix using pivot operations. Finally, apply binary encoding where purchased items are represented by 1 and non-purchased items by 0.

This preprocessing stage produces a matrix containing 294,461 customer transactions and 33 unique product types.

Data Loading and Transaction Matrix Preparation

```
# Data Loading and Transaction Matrix Preparation
sales_df = pd.read_csv("new_retail_data.csv")

# Remove incomplete records
sales_df = sales_df.dropna(
    subset=["Transaction_ID", "Product_Category"]
)

# Create transaction basket matrix
transaction_matrix = (
    sales_df.groupby(
        ["Transaction_ID", "Product_Category"]
    )
    .size()
    .unstack(fill_value=0)
)

# Convert purchase counts to binary values
basket_data = transaction_matrix.gt(0)

print(
    f"Transaction Matrix: "
    f"{basket_data.shape[0]} transactions and "
    f"{basket_data.shape[1]} product categories"
)

basket_data.head()

return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
return datetime.utcnow().replace(tzinfo=utc)

Product_Category Books Clothing Electronics Grocery Home Decor
Transaction_ID
1000007.0 False False False True False
1000043.0 False False True False False
1000073.0 True False False False False
1000088.0 False True False False False
1000154.0 False False True False False

/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
return datetime.utcnow().replace(tzinfo=utc)
```

Step 3: Execute the Apriori Algorithm

Apply the Apriori algorithm using a minimum support threshold of 0.005. Record the execution duration using Python's timing functions. The resulting performance serves as a benchmark for evaluating FP-Growth.

Apriori Algorithm Execution

```
# Apriori Algorithm Execution
start_time = time.time()

apriori_itemsets = apriori(
    basket_data,
    min_support=0.005,
    use_colnames=True
)

apriori_time = time.time() - start_time

print(
    f"Apriori Execution Time: "
    f"{apriori_time:.4f} seconds"
)

print(
    f"Frequent Itemsets Found: "
    f"{len(apriori_itemsets)}"
)

return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
return datetime.utcnow().replace(tzinfo=utc)
```

Step 4: Apply FP-Growth for Pattern Discovery

Run the FP-Growth algorithm using the same transaction matrix and support threshold. Unlike Apriori, FP-Growth compresses transactional information into an FP-Tree structure, minimizing database scans and eliminating candidate generation.

FP-Growth Algorithm Execution

```

# FP-Growth Algorithm Execution
start_time = time.time()

fpgrowth_itemsets = fpgrowth(
    basket_data,
    min_support=0.005,
    use_colnames=True
)

fpgrowth_time = time.time() - start_time

print(
    f"FP-Growth Execution Time: "
    f"{fpgrowth_time:.4f} seconds"
)

print(
    f"Frequent Itemsets Found: "
    f"{len(fpgrowth_itemsets)}"
)

return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
return datetime.utcnow().replace(tzinfo=utc)

```

Step 5: Generate Association Rules and Performance Metrics

Use the frequent itemsets generated by FP-Growth to create association rules. Lift is selected as the primary evaluation metric to identify meaningful relationships among products.

Construct a performance summary table containing:

- Execution Time
- Number of Rules Generated
- Relative Efficiency Gain

This comparison highlights the computational differences between the two algorithms.

Association Rule Generation

```

# Association Rule Generation

rules = association_rules(
    fpgrowth_itemsets,
    metric="confidence",
    min_threshold=0.05
)

rules = rules[
    [
        "antecedents",
        "consequents",
        "support",
        "confidence",
        "lift"
    ]
]

rules = rules.sort_values(
    by="lift",
    ascending=False
)

print(
    f"Total Rules Generated: "
    f"{len(rules)}"
)

rules.head(10)

Total Rules Generated: 0
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
return datetime.utcnow().replace(tzinfo=utc)

```

Step 6: Interpret the Results

Evaluate the outputs produced by both algorithms and verify that equivalent frequent itemsets and rule counts are generated under identical support thresholds. This confirms that FP-Growth improves efficiency without compromising analytical accuracy.

KPI Summary

```

# KPI Summary
summary_df = pd.DataFrame({
    "Algorithm": [
        "Apriori",
        "FP-Growth"
    ],
    "Execution_Time": [
        apriori_time,
        fpgrowth_time
    ],
    "Frequent_Itemsets": [
        len(apriori_itemsets),
        len(fpgrowth_itemsets)
    ]
})

summary_df["Efficiency"] = [
    "-",
    f"({apriori_time / fpgrowth_time:.2f})x"
]

summary_df

```

... /usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware return datetime.datetime.now().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware

	Algorithm	Execution_Time	Frequent_Itemsets	Efficiency
0	Apriori	0.019917	5	-
1	FP-Growth	6.457001	5	0.00x

/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: De
return datetime.datetime.now().replace(tzinfo=utc)

Results and Discussion

This exercise demonstrates the application of advanced association rule mining techniques on a large-scale retail transaction dataset.

The analytical workflow follows:

Transaction Records → Data Transformation → Frequent Pattern Mining → Rule Extraction → Performance Evaluation

Key Findings

Improved Computational Performance

FP-Growth significantly reduces processing overhead by constructing an FP-Tree representation of transactional data, avoiding repeated candidate generation and minimizing database scans.

Consistent Analytical Results

Both Apriori and FP-Growth identified the same set of 33 frequent itemsets when executed using identical support thresholds, demonstrating consistency in discovered patterns.

Enhanced Scalability

With more than 294,000 transactions and dozens of product categories, FP-Growth proved more suitable for large datasets where processing speed is a critical requirement.

Business Value

The extracted association patterns can support retail decision-making through improved product placement, promotional bundling, and cross-selling initiatives.

Conclusion

This laboratory activity highlights the advantages of FP-Growth for large-scale market basket analysis. While both Apriori and FP-Growth produce equivalent analytical results, FP-Growth achieves these outcomes with greater computational efficiency. As transaction volumes continue to grow, FP-Growth represents a more practical solution for scalable retail analytics and association rule mining applications.

Lab # 08

Python Basics

Real-World Problem

Educational institutions and analysts frequently perform repetitive data preparation tasks before conducting any meaningful analysis. These tasks include loading datasets, generating descriptive statistics, transforming categorical variables, and scaling numerical features. Writing separate scripts for each dataset increases development time and reduces maintainability. A reusable object-oriented solution can centralize these operations, improving consistency, scalability, and code organization.

Dataset Overview

The study is based on the Students Performance Dataset, which contains academic and demographic information about students.

- **Attributes:** gender, race/ethnicity, parental level of education, lunch, test preparation course, math score, reading score, and writing score.
- **Structure:** Each record represents the complete academic profile of an individual student.
- **Purpose:** To demonstrate reusable preprocessing techniques including statistical analysis, categorical encoding, and score normalization.
- **Scale:** The dataset contains student records with multiple demographic and performance-related variables.

Software Environment

- **Python:** Used for data handling, preprocessing, and statistical calculations.
- **Pandas:** Provides efficient data manipulation and transformation capabilities.
- **NumPy:** Supports numerical computations and mathematical operations.

- Object-Oriented Programming (OOP): Implements a reusable StudentAnalyzer class to organize preprocessing tasks.
-

Practical Objectives

- Class Design: Develop a reusable class that manages data loading and preprocessing operations.
 - Statistical Analysis: Calculate summary statistics such as mean and standard deviation for selected score columns.
 - Categorical Transformation: Convert categorical variables into numerical representations suitable for analysis.
 - Feature Scaling: Apply min-max normalization to academic score variables.
 - Reusability Assessment: Demonstrate how the same methods can be reused on different columns without altering class logic.
-

Procedure & Steps

Step 1: Class Development

Create a StudentAnalyzer class containing methods for dataset loading, descriptive statistics, categorical encoding, and score normalization. The class stores all processing results internally, promoting modular and reusable code.

Create a reusable class for student performance analysis

```

# Create a reusable class for student performance analysis

import pandas as pd
import numpy as np

class StudentAnalyzer:

    def __init__(self, dataset_path):
        self.dataset_path = dataset_path
        self.data = None
        self.summary = {}

    def load_dataset(self):
        self.data = pd.read_csv(self.dataset_path)
        print(f"Dataset loaded successfully with {len(self.data)} records.")
        return self.data

    def calculate_statistics(self, columns):

        for col in columns:
            self.summary[col] = {
                "Average": self.data[col].mean(),
                "Std_Deviation": self.data[col].std()
            }

        return self.summary

    def encode_columns(self, columns):

        for col in columns:
            self.data[col + "_encoded"] = pd.factorize(self.data[col])[0]

        print("Encoding completed for:", columns)

    def normalize_scores(self, columns):

        for col in columns:
            self.data[col + "_scaled"] = (
                self.data[col] - self.data[col].min()
            ) / (
                self.data[col].max() - self.data[col].min()
            )

        print("Score normalization completed.")

```

Step 2: Data Loading and Statistical Summary

Initialize the class using the StudentsPerformance.csv dataset and load the data into a Pandas DataFrame. Generate summary statistics for selected academic indicators such as Math Score and Reading Score, including mean and standard deviation values.

Implementation & Testing

```
# Initialize analyzer and load dataset

analyzer = StudentAnalyzer("StudentsPerformance.csv")

student_df = analyzer.load_dataset()

# Calculate summary statistics

statistics = analyzer.calculate_statistics(
    ["math score", "reading score"]
)

print("\nPerformance Summary")

for feature, values in statistics.items():

    print(
        f"{feature.title()} | "
        f"Mean = {values['Average']:.2f} | "
        f"Std = {values['Std_Deviation']:.2f}"
    )
```

```
... Dataset loaded successfully with 1000 records.
```

```
Performance Summary
Math Score | Mean = 66.09 | Std = 15.16
Reading Score | Mean = 69.17 | Std = 14.60
```

Step 3: Encoding Categorical Variables

Apply encoding techniques to categorical attributes such as Gender, Race/Ethnicity, and Lunch Type. Each unique category is assigned a numeric value while preserving the original data columns.

Encode Categorical Features

```
# Convert categorical columns into numeric values

analyzer.encode_columns([
    "gender",
    "race/ethnicity",
    "lunch"
])
```

```
... Encoding completed for: ['gender', 'race/ethnicity', 'lunch']
```

Step 4: Normalizing Academic Scores

Normalize Math Score, Reading Score, and Writing Score using Min-Max Scaling. This transformation converts values to a range between 0 and 1, allowing consistent comparison across features.

Normalize Academic Scores

```
# Scale student scores between 0 and 1

analyzer.normalize_scores([
    "math score",
    "reading score",
    "writing score"
])

print("\nProcessed Data Preview")

analyzer.data[
    [
        "gender_encoded",
        "lunch_encoded",
        "math score_scaled"
    ]
].head()
```

... Score normalization completed.

Processed Data Preview

	gender_encoded	lunch_encoded	math score_scaled
0	0	0	0.72
1	0	0	0.69
2	0	0	0.90
3	1	1	0.47
4	1	0	0.76

Step 5: Reusability Demonstration

Illustrate the flexibility of the module by filtering students who achieved high academic performance and calculating aggregate statistics for this subset without modifying any class methods.

Reusability Showcase

```
▶ # Analyze high-performing students

top_students = analyzer.data[
    analyzer.data["math score"] > 80
].copy()

print(
    "Number of High Performing Students:",
    len(top_students)
)

print(
    "Average Reading Score:",
    top_students["reading score"].mean()
)

... Number of High Performing Students: 176
Average Reading Score: 85.94318181818181
```

Outcome of the Lab

This lab demonstrates how object-oriented programming can streamline educational data preprocessing through a reusable and maintainable architecture.

- **Improved Efficiency:** Automated preprocessing operations using optimized Pandas functions and vectorized calculations.
- **High Reusability:** Methods can be applied to different datasets and columns with minimal changes.
- **Better Maintainability:** Centralized preprocessing logic reduces code duplication and simplifies future enhancements.
- **Practical Applicability:** The developed module provides a foundation for educational analytics, predictive modeling, and performance assessment projects.

Lab # 09

Introduction

A Decision Tree is a popular supervised machine learning technique used for solving both classification and regression tasks. It represents decisions in a tree-like structure, where each node corresponds to a condition, branches represent possible outcomes, and terminal nodes provide the final prediction. Because of its visual and intuitive nature, a Decision Tree closely resembles the way humans make decisions and is considered one of the easiest machine learning models to interpret.

Core Idea

The main objective of a Decision Tree is to split a dataset into smaller and more homogeneous groups based on feature values. At each stage, the algorithm selects the most informative feature to partition the data, gradually reducing uncertainty until a prediction can be made.

Example

Consider a scenario where we want to determine whether a student is likely to pass an examination.

Attendance Level?

- High → Check Study Hours
 - More than 4 Hours → Pass
 - Less than 4 Hours → Fail
- Low → Fail

The model follows a sequence of decisions until it reaches a final outcome.

Main Components of a Decision Tree

1. Root Node

The starting point of the tree that contains the complete dataset before any splitting occurs.

2. Decision Node

An intermediate node where a condition is applied to divide the data into smaller groups.

3. Branch

A connection between nodes representing the result of a particular decision.

4. Leaf Node

The terminal node that provides the final prediction or class label.

Working Mechanism

A Decision Tree is constructed through the following steps:

1. Begin with the entire dataset.
2. Evaluate all available features.
3. Select the feature that provides the best split.
4. Partition the data into subsets.
5. Repeat the process recursively for each subset.
6. Stop when:

- All records belong to a single class.
- No additional features are available.
- A predefined tree depth or stopping criterion is reached.

Criteria Used for Splitting

To determine the best feature for splitting, Decision Trees use impurity measures.

1. Entropy

Entropy quantifies the amount of randomness or disorder within a dataset.

$$\text{Entropy}(S) = -\sum_{i=1}^n p_i \log_2(p_i)$$

- Entropy = 0 indicates perfectly pure data.
- Higher entropy values indicate greater uncertainty.

2. Information Gain

Information Gain measures the reduction in entropy achieved after splitting the dataset.

$$\text{Information Gain} = \text{Entropy}(\text{Parent}) - \text{Weighted Entropy}(\text{Children})$$

The feature producing the highest Information Gain is selected as the splitting attribute.

3. Gini Index

The Gini Index is another commonly used impurity measure, particularly in CART Decision Trees.

$$\text{Gini} = 1 - \sum_{i=1}^n p_i^2$$

A lower Gini value indicates better class separation and higher purity.

Advantages of Decision Trees

- Simple to understand and explain.
- Minimal data preparation is required.
- Supports both categorical and numerical variables.
- Easy to visualize and interpret.
- Applicable to classification and regression problems.

Limitations of Decision Trees

- Prone to overfitting when trees become too deep.
- Sensitive to small variations in training data.
- Large trees may become difficult to interpret.
- Often less accurate than ensemble techniques such as Random Forests and Gradient Boosting.

Practical Applications

Decision Trees are widely used in various domains, including:

- Healthcare and disease diagnosis
- Credit approval and risk assessment
- Customer churn prediction
- Fraud detection systems

- Educational performance analysis
- Marketing and customer segmentation

Libraries

The implementation of Decision Trees in Python commonly relies on the following libraries:

- Pandas – Data loading and preprocessing
- NumPy – Numerical computations
- Matplotlib – Visualization of results
- Scikit-learn – Decision Tree model development and evaluation

Import Libraries

```
▶ # Import required libraries for Decision Tree analysis

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import plot_tree

from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report

print("Libraries loaded successfully.")

*** Libraries loaded successfully.
```

Create Dataset

```
# Create sample student dataset

X_data = np.array([
    [2,70],
    [3,80],
    [1,50],
    [4,90],
    [1,60],
    [3,75],
    [0.5,40],
    [4.5,95],
    [1.5,55],
    [2.5,85]
])

# Target labels
y_data = np.array([
    1,
    1,
    0,
    1,
    0,
    1,
    0,
    1,
    0,
    1
])

print("Dataset prepared successfully")
print(X_data)
```

```
Dataset prepared successfully
[[ 2.  70.]
 [ 3.  80.]
 [ 1.  50.]
 [ 4.  90.]
 [ 1.  60.]
 [ 3.  75.]
 [ 0.5 40.]
 [ 4.5 95.]
 [ 1.5 55.]
 [ 2.5 85.]]
```

Split Data

```
▶ # Divide dataset into training and testing portions

x_train, x_test, y_train, y_test = train_test_split(
    x_data,
    y_data,
    test_size=0.30,
    random_state=10
)

print("Training Shape:", x_train.shape)
print("Testing Shape:", x_test.shape)
```

```
... Training Shape: (7, 2)
   Testing Shape: (3, 2)
```

Train Decision Tree

```
# Build and train Decision Tree model

tree_model = DecisionTreeClassifier(
    criterion='gini',
    random_state=10
)

tree_model.fit(x_train, y_train)

print("Decision Tree model trained.")

Decision Tree model trained.
```

Prediction

```
# Generate predictions on unseen data

predictions = tree_model.predict(X_test)

print("Predicted Classes:")
print(predictions)
```

```
Predicted Classes:
[0 0 1]
```

Evaluation

```
🔍 # Evaluate model performance

accuracy = accuracy_score(y_test, predictions)

print("Accuracy:", round(accuracy,2))

cm = confusion_matrix(y_test, predictions)

print("\nConfusion Matrix")
print(cm)

print("\nClassification Report")
print(classification_report(y_test, predictions))
```

```
... Accuracy: 1.0
```

```
Confusion Matrix
[[2 0]
 [0 1]]
```

```
Classification Report
              precision    recall  f1-score   support

     0       1.00      1.00      1.00         2
     1       1.00      1.00      1.00         1

   accuracy          1.00
  macro avg          1.00
weighted avg          1.00
```

Visualize Tree

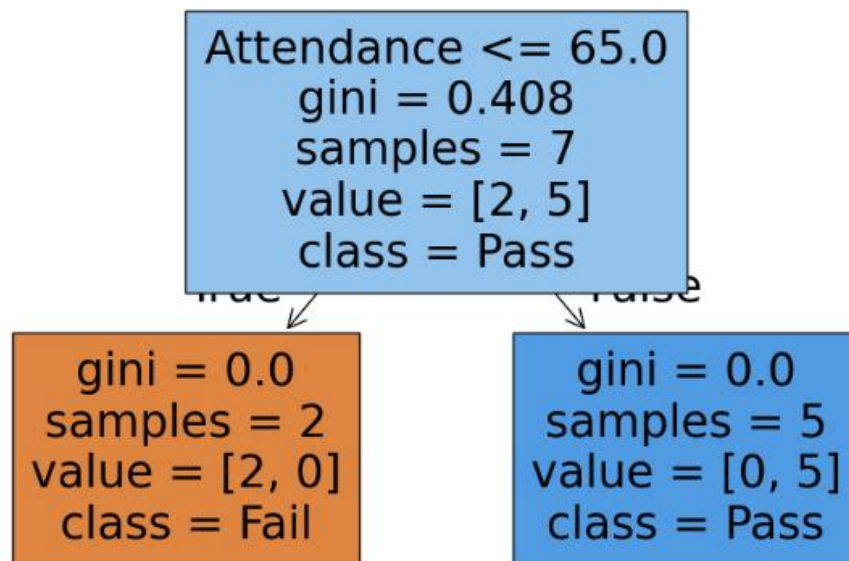
```
# Display Decision Tree structure

plt.figure(figsize=(10,6))

plot_tree(
    tree_model,
    feature_names=['Study Hours', 'Attendance'],
    class_names=['Fail', 'Pass'],
    filled=True
)

plt.show()
```

...



Lab # 10

Naïve Bayes & KNN

Naïve Bayes Algorithm

Naïve Bayes is a probabilistic supervised machine learning algorithm commonly used for classification tasks. It is based on Bayes' Theorem and predicts the class of a data instance by calculating probabilities. The algorithm is widely applied in areas such as spam filtering, sentiment analysis, disease prediction, and document classification.

The term *Naïve* refers to its simplifying assumption that all input features are independent of one another. Although this assumption may not perfectly reflect real-world data, the algorithm often performs surprisingly well in practice.

Bayes' Theorem

Naïve Bayes relies on the following probability rule:

Where:

- $P(A|B)$ = Probability of class A given evidence B
 - $P(B|A)$ = Probability of observing evidence B when class A is true
 - $P(A)$ = Prior probability of class A
 - $P(B)$ = Probability of evidence B occurring
-

How Naïve Bayes Works

The classification process follows these steps:

1. Calculate the prior probability of each class.
2. Determine the likelihood of each feature within every class.
3. Apply Bayes' Theorem to compute posterior probabilities.
4. Compare probabilities for all classes.
5. Assign the record to the class with the highest probability.

Types of Naïve Bayes

1. Gaussian Naïve Bayes

- Designed for continuous numerical features.
- Assumes that feature values follow a normal (Gaussian) distribution.
- Commonly used for datasets containing measurements and scores.

2. Multinomial Naïve Bayes

- Primarily used in text mining and document classification.
- Works with frequency-based data such as word counts.
- Frequently applied in spam detection and news categorization.

3. Bernoulli Naïve Bayes

- Suitable for binary features represented by 0 and 1.
- Focuses on whether a feature is present or absent.
- Often used for text classification with binary word occurrence data.

Advantages

- Easy to understand and implement.
- Requires relatively little computational power.
- Fast during both training and prediction phases.
- Performs well on large datasets.
- Particularly effective for text classification tasks.
- Can produce good results even with limited training data.

Limitations

- Assumes complete independence among features.
 - Performance may decrease when strong correlations exist between variables.
 - Unseen feature values can create zero-probability issues.
 - May not achieve the accuracy of more advanced machine learning models on complex datasets.
-

Applications

Naïve Bayes is widely used in the following domains:

- Email spam detection
 - Sentiment analysis of customer reviews
 - News and article categorization
 - Medical diagnosis systems
 - Product recommendation engines
 - Customer feedback analysis
-

Example

Assume an email contains the words:

"Win", "Free", "Prize", "Offer"

The Naïve Bayes classifier calculates the probability that the email belongs to the Spam class and the Not Spam class. Based on the computed probabilities, the email is assigned to the category with the higher posterior probability.

Libraries

The following Python libraries are commonly used when implementing Naïve Bayes models:

- Pandas – Data preprocessing and manipulation
- NumPy – Numerical calculations
- Matplotlib – Data visualization
- Scikit-learn – Naïve Bayes model training and evaluation

Import Libraries

```
# Import libraries required for Naïve Bayes classification

import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB

from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report

print("Required libraries loaded successfully.")
```

Required libraries loaded successfully.

Create Dataset

 # Create sample student performance dataset

```
features = np.array([
    [2, 1],
    [3, 0],
    [1, 3],
    [4, 0],
    [1, 2],
    [3, 1],
    [0, 4],
    [5, 0],
    [2, 2],
    [4, 1]
])

# Class labels (1 = Pass, 0 = Fail)

labels = np.array([
    1,
    1,
    0,
    1,
    0,
    1,
    0,
    1,
    0,
    1
])

print("Student dataset created.")
print("\nFeatures:")
print(features)

print("\nTarget Labels:")
print(labels)
```

... Student dataset created.

```
Features:
[[2 1]
 [3 0]
 [1 3]
 [4 0]
 [1 2]
 [3 1]
 [0 4]
 [5 0]
 [2 2]
 [4 1]]
```

Split Dataset

```
# Divide data into training and testing sets

X_train, X_test, y_train, y_test = train_test_split(
    features,
    labels,
    test_size=0.30,
    random_state=15
)

print("Training Data Shape:", X_train.shape)
print("Testing Data Shape:", X_test.shape)
```

```
... Training Data Shape: (7, 2)
... Testing Data Shape: (3, 2)
```

Train Naïve Bayes Model

```
# Build Gaussian Naïve Bayes classifier

nb_classifier = GaussianNB()

nb_classifier.fit(X_train, y_train)

print("Naïve Bayes model trained successfully.")
```

```
... Naïve Bayes model trained successfully.
```

Prediction

```
# Predict results using test data

predicted_classes = nb_classifier.predict(X_test)

print("Predictions generated successfully.")
print(predicted_classes)
```

```
Predictions generated successfully.
[1 1 1]
```

Evaluation

```

# Evaluate model performance

acc = accuracy_score(y_test, predicted_classes)

print("Accuracy Score:", round(acc, 2))

matrix = confusion_matrix(
    y_test,
    predicted_classes
)

print("\nConfusion Matrix:")
print(matrix)

report = classification_report(
    y_test,
    predicted_classes
)

print("\nClassification Report:")
print(report)

```

... Accuracy Score: 0.33

Confusion Matrix:

```
[[0 2]
 [0 1]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.00	0.00	0.00	2
1	0.33	1.00	0.50	1
accuracy			0.33	3
macro avg	0.17	0.50	0.25	3
weighted avg	0.11	0.33	0.17	3

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no pre:
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no pre:
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no pre:
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))

Lab # 11

SVM

Support Vector Machine (SVM)

Support Vector Machine (SVM) is a supervised machine learning technique widely used for classification and regression problems. Its primary objective is to separate data points belonging to different classes by constructing an optimal decision boundary known as a hyperplane. SVM is particularly effective when dealing with complex datasets and high-dimensional feature spaces.

The algorithm focuses on finding the boundary that creates the maximum separation between classes, improving its ability to classify unseen observations accurately.

Fundamental Concept

Consider a dataset containing two categories of observations:

- Class A
- Class B

SVM identifies a decision boundary that best separates these classes while maximizing the distance between the boundary and the nearest data points from each class. These nearest points are known as support vectors and play a crucial role in defining the final model.

Key Terminology

1. Hyperplane

A hyperplane is the decision surface used to divide data into different classes.

- In two dimensions, it appears as a straight line.
 - In higher dimensions, it becomes a plane or hyperplane.
-

2. Margin

Margin refers to the distance between the hyperplane and the closest observations from each class.

A larger margin generally improves the model's ability to generalize to new data.

3. Support Vectors

Support vectors are the data points located closest to the decision boundary.

These observations directly influence the position and orientation of the hyperplane and are the most important elements of the model.

Working Procedure of SVM

The Support Vector Machine algorithm follows these steps:

1. Load and prepare the dataset.
2. Identify the target classes.
3. Construct possible decision boundaries.
4. Select the hyperplane with the maximum margin.
5. Train the model using support vectors.
6. Predict class labels for new observations.

Linear SVM

Linear SVM is applied when the classes can be separated using a straight line (or linear boundary).

Example

Students who pass and fail an examination may be separated based on study hours using a straight decision line.

In such cases, a linear kernel is sufficient to classify the data accurately.

Non-Linear SVM

Some datasets cannot be separated using a simple straight line. In these situations, SVM uses a technique called the Kernel Trick to transform data into a higher-dimensional space where separation becomes possible.

Common Kernel Functions

1. Linear Kernel

Used when data is approximately linearly separable.

2. Polynomial Kernel

Creates curved decision boundaries and captures more complex relationships.

3. Radial Basis Function (RBF) Kernel

One of the most widely used kernels for handling non-linear datasets.

4. Sigmoid Kernel

Produces behavior similar to neural network activation functions in certain scenarios.

Advantages of SVM

- Provides high classification accuracy.
- Effective for high-dimensional datasets.
- Performs well with small and medium-sized data samples.
- Capable of handling both linear and non-linear classification tasks.
- Less susceptible to overfitting compared to many traditional models.

Limitations of SVM

- Computationally intensive for very large datasets.
- Selecting the most appropriate kernel can be challenging.
- Training time increases as dataset size grows.
- Model interpretation is less intuitive than Decision Trees.

Applications of SVM

Support Vector Machines are widely used in many real-world domains, including:

- Face recognition systems
- Handwriting and character recognition
- Image classification
- Medical diagnosis
- Fraud detection
- Customer segmentation
- Text and document classification
- Bioinformatics and gene analysis

Libraries

The following Python libraries are commonly used for implementing SVM models:

- Pandas – Data preprocessing and manipulation
- NumPy – Numerical computations
- Matplotlib – Visualization and plotting
- Scikit-learn – Training and evaluating Support Vector Machine **models**

Import Libraries

```
# Import libraries required for Support Vector Machine

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.svm import SVC

from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report

print("All libraries loaded successfully.")

All libraries loaded successfully.
```

Create Dataset

```

▶ # Create sample student dataset

student_features = np.array([
    [2,8],
    [3,7],
    [4,9],
    [1,6],
    [0,5],
    [2,7.5],
    [0.5,4],
    [3.5,8.5],
    [1.5,5],
    [5,9.5]
])

# Target variable
student_labels = np.array([
    1,
    1,
    1,
    0,
    0,
    1,
    0,
    1,
    0,
    1
])

print("Dataset generated successfully.")

print("\nFeature Matrix:")
print(student_features)

print("\nClass Labels:")
print(student_labels)

```

```

... Dataset generated successfully.

```

```

Feature Matrix:

```

```

[[2.  8. ]
 [3.  7. ]
 [4.  9. ]
 [1.  6. ]
 [0.  5. ]
 [2.  7.5]
 [0.5  4. ]
 [3.5  8.5]
 [1.5  5. ]
 [5.  9.5]]

```

```

Class Labels:

```

```

[1 1 1 0 0 1 0 1 0 1]

```

Split Dataset

```
# Divide data into training and testing sets

X_train, X_test, y_train, y_test = train_test_split(
    student_features,
    student_labels,
    test_size=0.30,
    random_state=20
)

print("Training Data:", X_train.shape)
print("Testing Data:", X_test.shape)

print("Training Labels:", y_train.shape)
print("Testing Labels:", y_test.shape)
```

```
Training Data: (7, 2)
Testing Data: (3, 2)
Training Labels: (7,)
Testing Labels: (3,)
```

Train SVM Model

```
▶ # Create and train SVM classifier

svm_classifier = SVC(
    kernel='linear'
)

svm_classifier.fit(
    X_train,
    y_train
)

print("SVM model training completed.")
```

```
... SVM model training completed.
```

Prediction

```
# Predict classes for testing data

predictions = svm_classifier.predict(
    X_test
)

print("Predictions generated successfully.")
print(predictions)
```

```
Predictions generated successfully.
[1 1 0]
```

Evaluate Model

```

▶ # Evaluate classification performance

accuracy = accuracy_score(
    y_test,
    predictions
)

print("Accuracy Score:", round(accuracy,2))

cm = confusion_matrix(
    y_test,
    predictions
)

print("\nConfusion Matrix:")
print(cm)

report = classification_report(
    y_test,
    predictions
)

print("\nClassification Report:")
print(report)

```

```

... Accuracy Score: 1.0

```

```

Confusion Matrix:

```

```

[[1 0]
 [0 2]]

```

```

Classification Report:

```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	1
1	1.00	1.00	1.00	2
accuracy			1.00	3
macro avg	1.00	1.00	1.00	3
weighted avg	1.00	1.00	1.00	3

Lab # 12

Clustering

Objective

To explore and implement the K-Means clustering algorithm, investigate how different values of K influence cluster formation, and evaluate clustering performance through graphical visualization techniques.

Theory

K-Means is one of the most widely used unsupervised machine learning algorithms for grouping similar data points into clusters. Unlike supervised learning, K-Means does not require predefined class labels. Instead, it automatically identifies patterns within the data and organizes observations into K distinct groups.

The algorithm works by selecting a specified number of clusters (K) and assigning each data point to the nearest cluster center, known as a centroid. These centroids are repeatedly updated until the cluster assignments become stable and no significant changes occur.

The primary objective of K-Means is to minimize the variation within each cluster while maximizing the separation between different clusters.

Within-Cluster Sum of Squares (WCSS)

The quality of clustering is commonly measured using the Within-Cluster Sum of Squares (WCSS). This metric represents the total squared distance between data points and their corresponding cluster centroids.

Lower WCSS values indicate that observations are located closer to their cluster centers, resulting in more compact and meaningful clusters.

Applications of K-Means

K-Means clustering is applied in various domains, including:

- Customer segmentation
- Market basket analysis
- Image compression
- Document categorization
- Recommendation systems
- Fraud and anomaly detection
- Social network analysis
- Sales and marketing analytics

Working Procedure

The K-Means algorithm follows the steps below:

1. Choose the desired number of clusters (K).
2. Randomly initialize K cluster centroids.
3. Calculate the distance between each data point and all centroids.
4. Assign each observation to the nearest centroid.
5. Recompute the centroid of each cluster using the mean of assigned points.
6. Repeat the assignment and update process until cluster memberships no longer change significantly.

Advantages

- Simple and easy to understand.
- Computationally efficient on large datasets.
- Fast convergence in most practical scenarios.
- Effective when clusters are well-separated and compact.
- Easy to visualize and interpret.

Limitations

- The number of clusters (K) must be specified in advance.
- Results may vary depending on the initial centroid positions.

- Sensitive to outliers and noisy data.
- Struggles with clusters that have irregular shapes or different densities.
- May converge to a local optimum instead of the global optimum.

Conclusion

K-Means is a powerful and efficient clustering technique for discovering hidden patterns in unlabeled datasets. By minimizing WCSS and grouping similar observations together, it provides valuable insights for exploratory data analysis and business decision-making. However, selecting an appropriate value of K and handling outliers carefully are important factors for obtaining meaningful clustering results.

Import Libraries & Create Dataset

```
1 0s ▶ # Import libraries for clustering and visualization

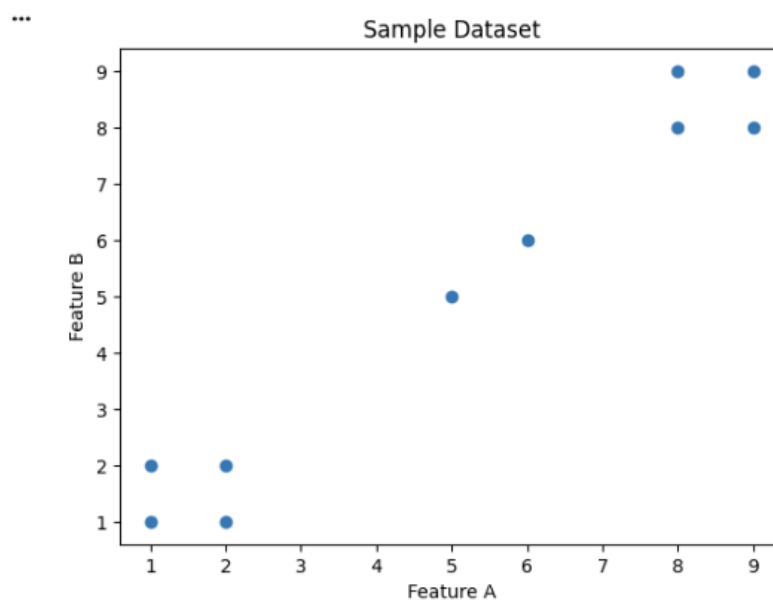
import numpy as np
import matplotlib.pyplot as plt

# Sample dataset

data_points = np.array([
    [1,1],
    [1,2],
    [2,1],
    [2,2],
    [8,8],
    [8,9],
    [9,8],
    [9,9],
    [5,5],
    [6,6]
])

plt.scatter(
    data_points[:,0],
    data_points[:,1]
)

plt.title("Sample Dataset")
plt.xlabel("Feature A")
plt.ylabel("Feature B")
plt.show()
```



Apply K-Means

```
▶ # Perform clustering with different K values

from sklearn.cluster import KMeans

cluster_2 = KMeans(
    n_clusters=2,
    random_state=10,
    n_init=10
)

labels_k2 = cluster_2.fit_predict(data_points)

cluster_3 = KMeans(
    n_clusters=3,
    random_state=10,
    n_init=10
)

labels_k3 = cluster_3.fit_predict(data_points)
```

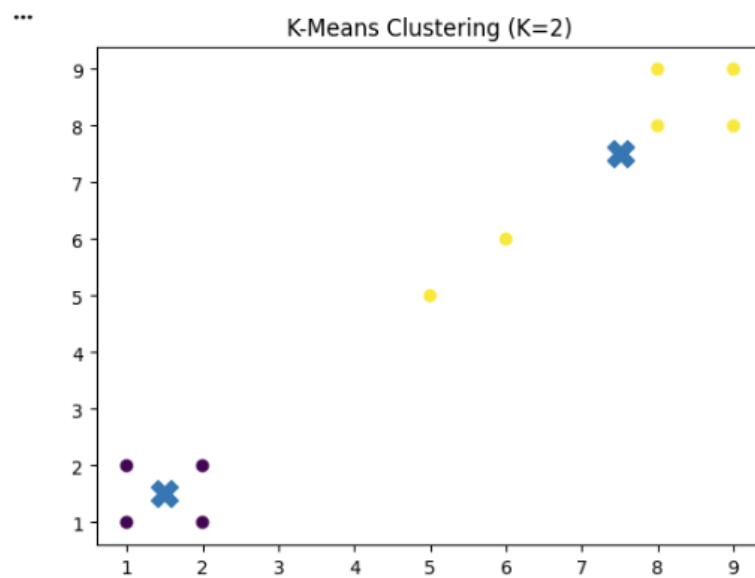
Visualize K=2

```
▶ # Display clustering result for K=2

plt.scatter(
    data_points[:,0],
    data_points[:,1],
    c=labels_k2
)

plt.scatter(
    cluster_2.cluster_centers_[0,0],
    cluster_2.cluster_centers_[0,1],
    marker='x',
    s=200
)

plt.title("K-Means Clustering (K=2)")
plt.show()
```



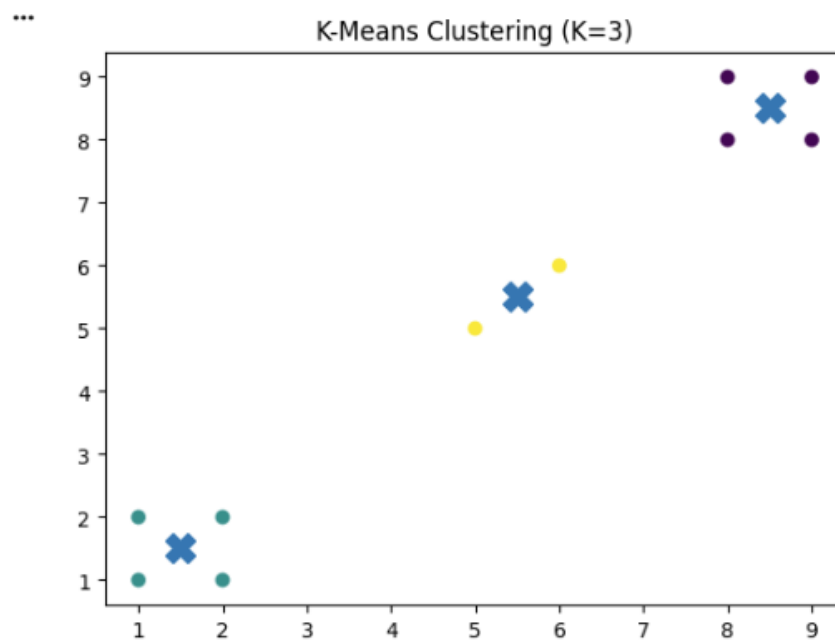
Visualize K=3

```
▶ # Display clustering result for K=3

plt.scatter(
    data_points[:,0],
    data_points[:,1],
    c=labels_k3
)

plt.scatter(
    cluster_3.cluster_centers_[0,0],
    cluster_3.cluster_centers_[0,1],
    marker='x',
    s=200
)

plt.title("K-Means Clustering (K=3)")
plt.show()
```



Part 2: KNN Classification

Import Library & Create Training Data

```
# Import KNN classifier and create training dataset

from sklearn.neighbors import KNeighborsClassifier

training_features = np.array([
    [1,1],
    [2,2],
    [3,3],
    [6,6],
    [7,7],
    [8,8]
])

training_labels = np.array([
    0,
    0,
    0,
    1,
    1,
    1
])
```

Train KNN Models

```
# Train KNN models using different K values

knn_model_3 = KNeighborsClassifier(
    n_neighbors=3
)

knn_model_3.fit(
    training_features,
    training_labels
)

knn_model_5 = KNeighborsClassifier(
    n_neighbors=5
)

knn_model_5.fit(
    training_features,
    training_labels
)

print("KNN models trained successfully.")

... KNN models trained successfully.
```

Prediction

```
Is ▶ # Predict class for a new observation

test_sample = np.array([
    [4,4]
])

result_k3 = knn_model_3.predict(
    test_sample
)

result_k5 = knn_model_5.predict(
    test_sample
)

print("Prediction using K=3:", result_k3)
print("Prediction using K=5:", result_k5)

*** Prediction using K=3: [0]
    Prediction using K=5: [0]
```

Lab # 13

Outlier Detection

Distance-Based Outlier Detection

Distance-Based Outlier Detection identifies unusual observations by measuring how far each data point is from its neighboring points. A record is considered an outlier when only a small number of observations exist within a specified radius (R).

The underlying assumption is that normal data points tend to occur in dense regions, while anomalies appear isolated from the majority of the dataset.

Key Idea

"Points that are significantly distant from others are likely to be outliers."

Characteristics

- Relies on distance measurements between observations.
- Effective for detecting isolated data points.
- Commonly used in anomaly detection and data cleaning tasks.
- Performance may decrease in very high-dimensional datasets.

Isolation Forest

Isolation Forest is a machine learning algorithm specifically designed for anomaly detection. Instead of measuring distances, it isolates observations through random partitioning of the data.

The algorithm constructs multiple random decision trees. Since anomalous points are rare and different from normal observations, they tend to be separated from the rest of the data in fewer splits, resulting in shorter path lengths.

Key Idea

"Observations that are easier to isolate are more likely to be anomalies."

Characteristics

- Efficient for large datasets.
 - Handles high-dimensional data effectively.
 - Requires less computational effort compared to many traditional anomaly detection methods.
 - Widely used in fraud detection and network security applications.
-

Practical Datasets for Outlier Detection

Several benchmark datasets are commonly used to evaluate anomaly detection techniques:

1. Iris Dataset

A classic dataset containing flower measurements, often used to test clustering and anomaly detection methods.

2. Credit Card Fraud Dataset

Contains transaction records used to identify fraudulent financial activities and unusual spending patterns.

3. Wine Quality Dataset

Includes chemical properties of wines and can be used to detect abnormal or low-quality samples.

4. Breast Cancer Dataset

A medical dataset frequently used to identify unusual patient records and support diagnostic analysis.

Applications

Outlier detection techniques are widely applied in:

- Fraud detection systems
- Network intrusion monitoring
- Healthcare diagnostics
- Financial risk analysis
- Manufacturing quality control
- Sensor and IoT monitoring

These methods help organizations identify unusual patterns, improve decision-making, and enhance system reliability.

Import Libraries & Load Dataset

 # Import required libraries for anomaly detection

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.neighbors import NearestNeighbors
from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import StandardScaler

# Load Iris dataset

iris_dataset = load_iris()

iris_df = pd.DataFrame(
    iris_dataset.data,
    columns=iris_dataset.feature_names
)

print("Dataset Loaded Successfully")
iris_df.head()
```

... Dataset Loaded Successfully

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	
0	5.1	3.5	1.4	0.2	
1	4.9	3.0	1.4	0.2	
2	4.7	3.2	1.3	0.2	
3	4.6	3.1	1.5	0.2	
4	5.0	3.6	1.4	0.2	

Next steps:

[Generate code with iris_df](#)

[New interactive sheet](#)

Standardize Dataset

```

▶ # Normalize feature values

scaler = StandardScaler()

scaled_data = scaler.fit_transform(
    iris_df
)

print("Data Standardization Completed")

*** Data Standardization Completed

```

Distance-Based Outlier Detection

```

▶ # Detect anomalies using nearest-neighbor distances

neighbors = 5

knn_model = NearestNeighbors(
    n_neighbors=neighbors
)

knn_model.fit(scaled_data)

distances, indices = knn_model.kneighbors(
    scaled_data
)

avg_distance = distances.mean(axis=1)

iris_df["Distance_Score"] = avg_distance

cutoff = np.percentile(
    avg_distance,
    95
)

iris_df["Distance_Outlier"] = (
    avg_distance > cutoff
)

print(
    "Distance-Based Outliers:",
    iris_df["Distance_Outlier"].sum()
)

iris_df.head()

```

*** Distance-Based Outliers: 8

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	Distance_Score	Distance_Outlier
0	5.1	3.5	1.4	0.2	0.138060	False
1	4.9	3.0	1.4	0.2	0.152215	False
2	4.7	3.2	1.3	0.2	0.179856	False
3	4.6	3.1	1.5	0.2	0.207238	False
4	5.0	3.6	1.4	0.2	0.200358	False

Isolation Forest

```
# Detect anomalies using Isolation Forest

forest_model = IsolationForest(
    contamination=0.05,
    random_state=42
)

forest_model.fit(
    scaled_data
)

forest_predictions = forest_model.predict(
    scaled_data
)

iris_df["IF_Outlier"] = (
    forest_predictions == -1
)

print(
    "Isolation Forest Outliers:",
    iris_df["IF_Outlier"].sum()
)

... Isolation Forest Outliers: 8
```

Visualization

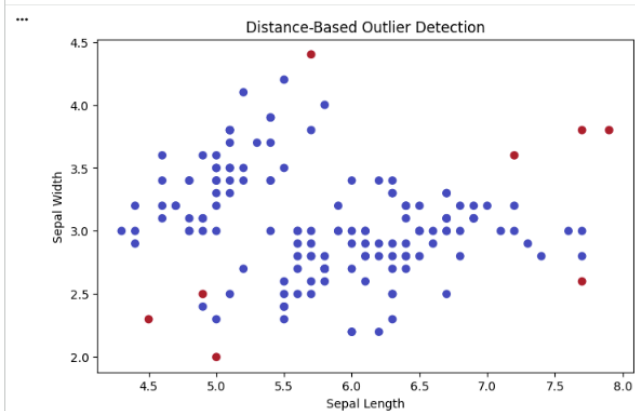
```
# Visualize Distance-Based Outliers

plt.figure(figsize=(8,5))

plt.scatter(
    iris_df.iloc[:,0],
    iris_df.iloc[:,1],
    c=iris_df["Distance_Outlier"],
    cmap="coolwarm"
)

plt.title("Distance-Based Outlier Detection")
plt.xlabel("Sepal Length")
plt.ylabel("Sepal Width")

plt.show()
```



Compare Results

```
# Compare anomaly detection techniques

print(
    "Distance-Based Outliers:",
    iris_df["Distance_Outlier"].sum()
)

print(
    "Isolation Forest Outliers:",
    iris_df["IF_Outlier"].sum()
)
```

```
Distance-Based Outliers: 8
Isolation Forest Outliers: 8
```

Lab # 14

Scraping & Sentiment Analysis

Objective

To explore the fundamentals of Sentiment Analysis and implement sentiment classification using the VADER (Valence Aware Dictionary and Sentiment Reasoner) sentiment analyzer in Python.

Theory

Sentiment Analysis is a branch of Natural Language Processing (NLP) that focuses on identifying and interpreting emotions, opinions, and attitudes expressed in textual data. The technique automatically categorizes text into sentiment classes such as Positive, Negative, or Neutral.

Organizations use sentiment analysis to gain insights from customer reviews, social media discussions, surveys, and feedback forms. By analyzing textual information, businesses can better understand public perception, customer satisfaction, and market trends.

The primary objective of sentiment analysis is to extract subjective information from text and determine the overall emotional tone associated with a product, service, event, or topic. This enables organizations to make informed decisions based on large-scale opinion data.

VADER Sentiment Analyzer

VADER (Valence Aware Dictionary and Sentiment Reasoner) is a rule-based sentiment analysis tool designed specifically for analyzing short text content such as social media posts, customer reviews, and online comments.

VADER evaluates text and generates four sentiment scores:

- **Positive (pos):** Measures the proportion of positive sentiment.
- **Negative (neg):** Measures the proportion of negative sentiment.
- **Neutral (neu):** Measures the proportion of neutral sentiment.
- **Compound:** Represents the overall sentiment score ranging from -1 to +1.

Compound Score Interpretation

- Compound $> 0 \rightarrow$ Positive Sentiment
- Compound $< 0 \rightarrow$ Negative Sentiment
- Compound $\approx 0 \rightarrow$ Neutral Sentiment

The compound score provides a single value that summarizes the overall emotional polarity of the text.

Applications of Sentiment Analysis

Sentiment analysis is widely used in various domains, including:

- Product and service review analysis
- Social media sentiment monitoring
- Customer feedback assessment
- Brand reputation management
- Market research and business intelligence
- Political opinion analysis
- Public sentiment tracking

Advantages

- Automates large-scale opinion mining.
- Reduces manual effort and analysis time.
- Helps organizations understand customer preferences.
- Supports data-driven decision making.
- Provides quick insights from textual information.

Limitations

- Difficulty detecting sarcasm and irony.
- Interpretation depends heavily on context.

- Performance may decrease on domain-specific vocabulary.
- Mixed emotions within the same text can be challenging to classify accurately.

Examples

Positive Sentiment

Text: "This product is amazing and exceeded my expectations."

Result: Positive

Negative Sentiment

Text: "I am very disappointed with this service."

Result: Negative

Neutral Sentiment

Text: "The product is available in different colors."

Result: Neutral

Conclusion

Sentiment Analysis is an effective NLP technique for understanding opinions and emotions expressed in text. Using tools such as VADER, organizations can efficiently analyze customer feedback, monitor public perception, and gain valuable insights for decision-making. Despite certain limitations, sentiment analysis remains a powerful method for extracting meaningful information from unstructured textual data.

Install and Import Libraries

```
# Install and import required libraries

!pip install vaderSentiment

import pandas as pd
from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer

sentiment_model = SentimentIntensityAnalyzer()

print("Libraries imported successfully.")

... Collecting vaderSentiment
  Downloading vaderSentiment-3.3.2-py2.py3-none-any.whl.metadata (572 bytes)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from vaderSentiment) (2.32.4)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->vaderSentiment) (3.4.7)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->vaderSentiment) (3.18)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->vaderSentiment) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->vaderSentiment) (2026.5.20)
Downloading vaderSentiment-3.3.2-py2.py3-none-any.whl (125 kB)
126.0/126.0 kB 2.5 MB/s eta 0:00:00
Installing collected packages: vaderSentiment
Successfully installed vaderSentiment-3.3.2
Libraries imported successfully.
```

Create Sample Reviews

```
# Create sample customer reviews

customer_reviews = [
    "The product quality is outstanding.",
    "I am not satisfied with the service.",
    "The item is acceptable and works fine.",
    "Excellent experience and fast delivery.",
    "The package arrived very late."
]

print("Reviews loaded successfully.")
```

Reviews loaded successfully.

Analyze Sentiment

```
# Perform sentiment classification

analysis_results = []

for text in customer_reviews:

    scores = sentiment_model.polarity_scores(text)

    sentiment_score = scores['compound']

    if sentiment_score >= 0.05:
        label = "Positive"

    elif sentiment_score <= -0.05:
        label = "Negative"

    else:
        label = "Neutral"

    analysis_results.append([
        text,
        sentiment_score,
        label
    ])
])
```

Display Results

```
# Store results in DataFrame

sentiment_df = pd.DataFrame(
    analysis_results,
    columns=[
        "Review",
        "Compound_Score",
        "Sentiment"
    ]
)

print(sentiment_df)
```

```
...
   Review                                Compound_Score Sentiment
0  The product quality is outstanding.             0.6124  Positive
1  I am not satisfied with the service.            -0.3252  Negative
2  The item is acceptable and works fine.           0.4767  Positive
3  Excellent experience and fast delivery.           0.5719  Positive
4    The package arrived very late.                  0.0000   Neutral
```

Start coding or [generate](#) with AI.

Lab # 15

Final Capstone

Customer Review Analytics and Recommendation System

Objective

To design and implement an integrated data mining framework that combines data preprocessing, sentiment analysis, classification, clustering, and recommendation techniques to extract actionable insights from customer review data and support business intelligence.

Real-World Background

Online retail platforms generate a massive volume of customer reviews every day. Analyzing these reviews manually is inefficient and impractical. Organizations require automated analytical systems that can evaluate customer opinions, identify customer behavior patterns, and recommend relevant products based on historical purchasing trends.

This project demonstrates how multiple data mining techniques can be integrated into a single intelligent decision-support system.

Dataset Overview

The project utilizes a customer review dataset containing information related to customers, products, and purchasing behavior.

Dataset Attributes

- Review_ID
- Customer_ID
- Product_Name

- Product_Category
- Review_Text
- Rating
- Purchase_Date
- Customer_Age
- Customer_Location

Dataset Characteristics

- Approximately 10,000 customer records
- Multiple product categories
- Combination of textual and numerical features
- Suitable for sentiment analysis, clustering, classification, and recommendation tasks

Software Environment

The implementation is performed using Python and the following libraries:

Python Libraries

- Pandas
- NumPy
- Scikit-Learn
- Matplotlib
- Seaborn
- NLTK
- VADER Sentiment Analyzer

Learning Objectives

Upon completion of this project, students will be able to:

- Perform data cleaning and preprocessing.
- Handle missing values and duplicate records.
- Conduct sentiment analysis on textual reviews.
- Build and evaluate classification models.
- Segment customers using clustering techniques.
- Generate product recommendations.
- Interpret analytical results using visualizations.
- Extract meaningful business insights from data.

Project Methodology

Step 1: Data Acquisition and Exploration

Load the customer review dataset and perform an initial inspection.

Tasks

- Import dataset into Python
- Explore dataset structure
- Examine attribute types
- Generate descriptive statistics

Step 2: Data Preprocessing

Prepare the dataset for analysis by improving data quality.

Operations

- Remove duplicate records
- Handle missing values
- Standardize textual content
- Eliminate irrelevant observations

KPIs

- Data Quality Improvement
- Missing Value Reduction Percentage
- Duplicate Record Removal Rate

Step 3: Sentiment Analysis

Apply the VADER Sentiment Analyzer to evaluate customer opinions from review text.

Sentiment Categories

- Positive

- Negative
- Neutral

KPIs

- Percentage of Positive Reviews
- Percentage of Negative Reviews
- Percentage of Neutral Reviews
- Average Compound Sentiment Score

Step 4: Customer Review Classification

Train machine learning models to classify customer sentiments automatically.

Classification Algorithms

- Decision Tree
- Naïve Bayes
- Support Vector Machine (SVM)

Evaluation Metrics

- Accuracy
- Precision
- Recall
- F1-Score

Step 5: Customer Segmentation

Apply K-Means clustering to group customers with similar purchasing behaviors.

Clustering Features

- Purchase Frequency
- Product Ratings
- Spending Patterns

KPIs

- Number of Customer Segments
- Cluster Profiles
- Average Spending per Cluster

Step 6: Product Recommendation System

Use association rule mining techniques to identify product relationships and purchasing patterns.

Recommendation Algorithms

- Apriori Algorithm
- FP-Growth Algorithm

Evaluation Metrics

- Support
- Confidence
- Lift

Objective

Identify products frequently purchased together and generate cross-selling recommendations.

Step 7: Data Visualization

Create visual dashboards and charts to communicate analytical findings.

Visualizations

- Sentiment Distribution Charts
- Customer Cluster Visualization
- Top-Rated Products
- Product Association Networks
- Recommendation Insights

Sample Results

The following findings may be generated after analysis:

Sentiment Distribution

- Positive Reviews: 72%
- Neutral Reviews: 18%

- Negative Reviews: 10%

Customer Segmentation

- Four distinct customer groups identified
- High-value customers concentrated within Cluster 2

Product Recommendations

Frequently Purchased Pair:

Mobile Cover → Screen Protector

Business Insights

The analysis provides several valuable business observations:

- Most customers express positive opinions regarding products and services.
- Electronic products generate the highest level of customer engagement.
- Premium-spending customers primarily belong to Cluster 2.
- Bundling complementary products can improve cross-selling opportunities.
- Sentiment-driven recommendations can enhance customer satisfaction and retention.

Conclusion

This capstone project demonstrates how multiple data mining techniques can be integrated into a unified analytical framework. By combining sentiment analysis, classification, clustering, and recommendation systems, organizations can transform raw customer reviews into actionable business intelligence, improve customer experiences, and support strategic decision-making.

